

Linux Kernel Programming — Hands-On Lab Manual



Prerequisites

- A Linux machine (Ubuntu 20.04+ / Debian 11+) — **native or VM** (not WSL for kernel work)
- **OR** a Raspberry Pi (3B+/4/5) running Raspberry Pi OS (Debian-based)
- Root/sudo access
- GCC, Make, kernel headers installed
- Basic C programming and Linux command-line familiarity
- At least 20 GB free disk space (for kernel source builds)

```
# Quick setup – run this first!
sudo apt update
sudo apt install -y build-essential gcc make libncurses-dev bison flex \
    libssl-dev libelf-dev bc dwarves cpio lz4 zstd \
    qemu-system-x86 qemu-utils debootstrap \
    git wget curl strace ltrace

# Install kernel headers for your running kernel
sudo apt install -y linux-headers-$(uname -r)
```

🍓 **Raspberry Pi Users:** Most module programming, `procfs/sysfs`, `timer`, `thread`, and character driver labs work directly on Raspberry Pi. Labs marked with 🍓 have Raspberry Pi-specific instructions. Install headers with: `sudo apt install raspberrypi-kernel-headers`

⚠️ **Important:** Kernel experiments can crash your system. Always work inside a **virtual machine** (VirtualBox, VMware, or QEMU) for safety, or use a dedicated Raspberry Pi for testing. Never test custom kernels on production machines.

LAB 1: Introduction to Linux Kernel



Mission: "Entering the Kernel World"

Story: You have been recruited by the OS Internals division. Before touching any kernel code, you must prove you understand the architecture — what the kernel does, where it lives, and how it differs from everything else running on the system. Your first briefing starts now.

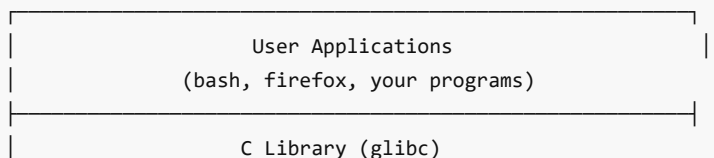


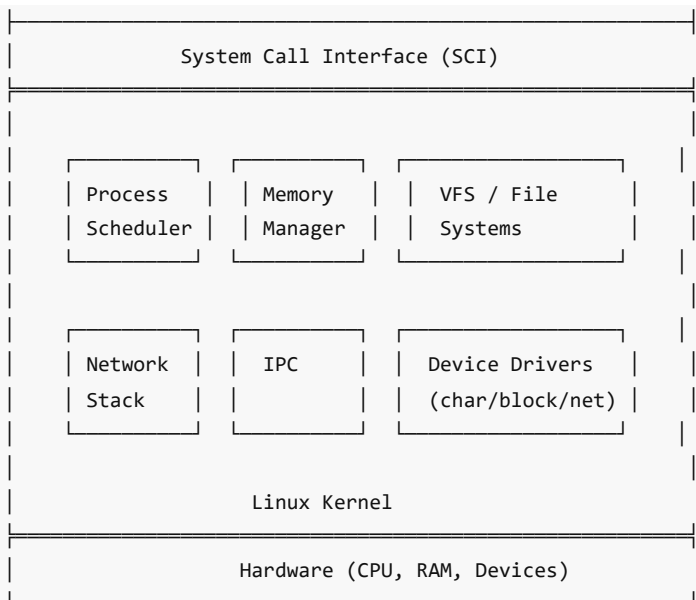
Background

The **Linux kernel** is the core of the operating system. It manages:

- **Process scheduling** — deciding which process runs on which CPU
- **Memory management** — virtual memory, paging, allocation
- **Device drivers** — interfacing with hardware
- **File systems** — organizing data on storage
- **Networking** — TCP/IP stack, sockets, routing
- **Security** — access control, capabilities, namespaces

The kernel is a **monolithic kernel with loadable module support**:





❏ Exercise 1.1: Explore Your Running Kernel

Objective: Discover information about the currently running kernel.

Step 1: Gather kernel information:

```
# Kernel version
uname -r
uname -a

# Kernel release info
cat /proc/version

# Kernel command line (how it was booted)
cat /proc/cmdline

# Kernel config (if available)
cat /boot/config-$(uname -r) | head -50

# Or from /proc
zcat /proc/config.gz 2>/dev/null | head -50
```

Step 2: Explore the kernel's view of the system:

```
# CPU info as kernel sees it
cat /proc/cpuinfo | grep "model name" | head -4

# Memory info
cat /proc/meminfo | head -10

# Loaded kernel modules
lsmod | head -20
lsmod | wc -l

# Kernel messages (ring buffer)
dmesg | tail -30
```

Step 3: Check kernel symbols:

```
# Kernel symbol table
sudo cat /proc/kallsyms | head -20
sudo cat /proc/kallsyms | wc -l

# How many exported symbols?
sudo cat /proc/kallsyms | grep " T " | wc -l
```

Step 4: Answer in `answers.txt` :

1. What kernel version are you running?
2. How many kernel modules are currently loaded?
3. How many symbols are in the kernel symbol table?
4. What was the kernel command line used to boot?

✅ **Checkpoint:** You should understand that the kernel is a running program with its own view of the system, accessible via `/proc` .

❏ Exercise 1.2: Kernel Space vs User Space

Objective: Understand the fundamental boundary between kernel and user space.

Step 1: Examine virtual memory layout:

```
# User process memory map
cat /proc/self/maps | head -20

# Where does kernel space start? (on x86_64, typically 0xffff...)
sudo cat /proc/kallsyms | head -5
```

Step 2: Navigate to the lab directory and examine the template:

```
cd code/kernel_programming/lab01_intro/
cat kernel_user_boundary.c
```

Step 3: Compile and run the user-space probe:

```
make
./kernel_user_boundary
```

This program demonstrates:

- User-space addresses (stack, heap, code)
- The system call boundary (how user code enters the kernel)
- Address space differences

Step 4: Try to read kernel memory from user space:

```
# This WILL fail – and that's the point!
sudo cat /proc/kallsyms | head -1 # Get a kernel address
# Trying to access it from user space = SEGVFAULT
```

Step 5: Observe system call transitions:

```
# Trace system calls made by a simple command
strace -c ls /tmp 2>&1 | tail -20

# See the actual kernel entry/exit
strace -e trace=write echo "Hello Kernel" 2>&1
```

 **Challenge:** Write a program that measures the overhead of a system call by timing 1 million `getpid()` calls vs 1 million function calls. What's the ratio?

🔗 **Exercise 1.3: Classification of Drivers**

Objective: Understand the three main categories of Linux device drivers.

 Background

Linux classifies devices into three fundamental types:

Type	Purpose	Device Files	Examples
Character	Byte-stream access (sequential)	/dev/tty*, /dev/null, /dev/random	Serial ports, keyboards, custom sensors
Block	Block-level access (random access, buffered)	/dev/sda*, /dev/nvme*, /dev/loop*	Hard disks, SSDs, USB storage
Network	Packet-based (no device file)	No /dev/ entry — uses sockets	eth0, wlan0, lo

```
Device Driver Types
├─ Character Drivers (c)
│  └─ Sequential byte stream
│  └─ Major/Minor numbers
│  └─ open, read, write, ioctl, close
├─ Block Drivers (b)
│  └─ Random access, fixed-size blocks
│  └─ I/O scheduler, buffer cache
│  └─ Uses request queues
└─ Network Drivers
   └─ No device node
   └─ Socket-based interface
   └─ sk_buff, NAPI, netdev
```

Step 1: Explore device files on your system:

```
# List character devices
ls -la /dev/ | grep "^c" | head -15

# List block devices
ls -la /dev/ | grep "^b" | head -15
lsblk

# Network interfaces
ip link show
ls /sys/class/net/
```

Step 2: Understand major/minor numbers:

```
# See major numbers and their assigned drivers
cat /proc/devices

# Character devices section
cat /proc/devices | head -30

# Block devices section
cat /proc/devices | tail -20

# Examine a specific device node
stat /dev/null
stat /dev/sda 2>/dev/null || stat /dev/vda 2>/dev/null
```

Step 3: Explore the driver hierarchy via sysfs:

```
# All registered devices
ls /sys/class/

# Block devices
ls /sys/class/block/

# Input devices (character)
ls /sys/class/input/ 2>/dev/null

# Network devices
ls /sys/class/net/

# Detailed driver info
cat /sys/class/net/lo/type
cat /sys/class/block/${lsblk -d -o NAME | tail -1}/size 2>/dev/null
```

Step 4: Answer in `answers.txt` :

1. How many character device major numbers are registered on your system?
2. What is the major/minor number of `/dev/null` ?
3. Name 3 character devices, 3 block devices, and 3 network interfaces on your system.
4. Why don't network devices have entries in `/dev/` ?

✓ **Checkpoint:** You should understand char vs block vs network drivers and how devices are represented in Linux.

LAB 2: Kernel Versioning & Source Tree

Mission: "Navigating the Kernel Codebase"

Story: Your team lead has asked you to prepare a patch for an upstream kernel. But first, you need to understand how the kernel is versioned, where to get the source, and how the massive codebase is organized. Millions of lines of code — you need a map.

Background

Kernel Version Format:

```
v<major>.<minor>.<patch>[-rc<N>][-<extra>]
```

Examples:

```
v6.8.0      - Stable release
v6.9-rc3    - Release candidate 3 for 6.9
v5.15.120   - Long-term stable (LTS) patch release
v6.8.0-42-generic - Distribution kernel (Ubuntu)
```

Release Cycle:

- New major release every ~2-3 months
- Merge window: 2 weeks after release (new features)
- RC phase: ~6-8 release candidates (bug fixes only)
- LTS kernels: maintained for 2-6 years

❏ Exercise 2.1: Understanding Kernel Versions

Objective: Parse and understand kernel version numbers.

Step 1: Check your current kernel version in multiple ways:

```
# Simple version
uname -r

# Full version string
uname -v

# Kernel release + machine
uname -rm

# Version from source config
cat /boot/config-$(uname -r) | grep "CONFIG_VERSION"

# Check Makefile (if source is installed)
head -5 /usr/src/linux-headers-$(uname -r)/Makefile 2>/dev/null
```

Step 2: Understand what each part means:

```
# Parse your kernel version
KVER=$(uname -r)
echo "Full version: $KVER"
echo "Major.Minor.Patch: $(echo $KVER | cut -d'-' -f1)"
echo "Local version: $(echo $KVER | cut -d'-' -f2-)"
```

Step 3: Check available LTS and stable kernels:

```
# See what kernel packages are available
apt list --installed 2>/dev/null | grep linux-image
apt list --all-versions linux-image-generic 2>/dev/null | head -10
```

Step 4: Answer in `answers.txt` :

1. What is the difference between `6.8.0` and `6.8.0-42-generic` ?
2. What does "LTS" mean and why does it matter for production?
3. What happens during the "merge window"?

❏ Exercise 2.2: Navigating the Kernel Source Tree

Objective: Download the kernel source and understand its directory structure.

Step 1: Get the kernel source:

```
# Option A: Install kernel source for your distro version
sudo apt install -y linux-source
ls /usr/src/linux-source-*

# Option B: Download a specific version from kernel.org
cd /tmp
wget https://cdn.kernel.org/pub/linux/kernel/v6.x/linux-6.8.tar.xz
tar xf linux-6.8.tar.xz
cd linux-6.8

# Option C: Shallow clone (faster, recommended for exercises)
cd /tmp
git clone --depth 1 https://git.kernel.org/pub/scm/linux/kernel/git/stable/linux.git
cd linux
```

Step 2: Explore the top-level directory structure:

```
ls -l

# Count total files
find . -type f | wc -l

# Count lines of code (this takes a while!)
find . -name "*.c" | xargs wc -l 2>/dev/null | tail -1
find . -name "*.h" | xargs wc -l 2>/dev/null | tail -1
```

Step 3: Understand each top-level directory:

Directory	Purpose	Key Contents
arch/	Architecture-specific code	x86, arm, arm64, riscv
block/	Block I/O layer	I/O schedulers, request queues
crypto/	Cryptographic API	AES, SHA, RSA implementations
Documentation/	Kernel docs	Admin guide, driver API, devicetree bindings
drivers/	All device drivers	The largest directory (~60% of kernel)
fs/	Filesystem implementations	ext4, btrfs, proc, sysfs
include/	Header files	UAPI headers, kernel headers
init/	Kernel initialization	main.c — where the kernel starts
ipc/	Inter-process communication	Shared memory, semaphores, msg queues
kernel/	Core kernel code	Scheduler, fork, signals, time
lib/	Kernel library routines	String ops, sort, CRC, compression
mm/	Memory management	Page allocator, slab, vmalloc, mmap
net/	Networking stack	TCP, UDP, IPv4/6, netfilter
scripts/	Build scripts and tools	Kconfig, checkpatch, cocci

security/	Security frameworks	SELinux, AppArmor, capabilities
sound/	Audio subsystem (ALSA)	Sound drivers, codec support
tools/	Userspace tools	perf, bpf, selftests
virt/	Virtualization support	KVM

```
# Explore key directories
ls arch/
ls drivers/ | head -20
ls fs/
ls net/

# The biggest directory
du -sh drivers/
find drivers/ -name "*.c" | wc -l

# Where does boot start?
head -30 init/main.c

# The very first function called
grep -n "start_kernel" init/main.c | head -5
```

Step 4: Explore key files:

```
# The kernel's Makefile
head -20 Makefile

# Kernel's main configuration symbol
grep "KCONFIG" Makefile | head -5

# MAINTAINERS – who maintains what
wc -l MAINTAINERS
grep -A 5 "MEMORY MANAGEMENT" MAINTAINERS | head -10

# Coding style
head -50 Documentation/process/coding-style.rst
```

 **Challenge:** Find the source file that implements the `fork()` system call. What function name does the kernel use internally? (Hint: look in `kernel/`)

LAB 3: Configuring & Building the Kernel

Mission: "Building Your Own Kernel"

Story: The standard kernel doesn't support a custom hardware device your team needs. You must learn to configure and build a custom kernel from source. This is a rite of passage for every kernel developer.

Background

The kernel build system uses **Kconfig** for configuration and **Kbuild** (an extension of Make) for building.

Configuration options:

- `y` — built-in (compiled into kernel image)
- `m` — module (compiled as loadable `.ko` file)
- `n` — disabled (not compiled at all)

Configuration tools:

```
make config      - line-by-line text prompts (tedious)
make menuconfig - ncurses TUI (most common)
make xconfig    - Qt GUI
make gconfig    - GTK GUI
make defconfig  - default config for architecture
make oldconfig  - update old config for new kernel
make localmodconfig - config based on loaded modules
```

❏ Exercise 3.1: Kernel Configuration

Objective: Learn to configure the kernel using menuconfig.

Step 1: Start from a default config:

```
cd /tmp/linux # or wherever you have the kernel source

# Start with architecture default
make defconfig

# See what was generated
ls -la .config
wc -l .config
grep "=y" .config | wc -l # Built-in options
grep "=m" .config | wc -l # Module options
grep "not set" .config | wc -l # Disabled options
```

Step 2: Explore menuconfig:

```
make menuconfig
```

Navigate the menus and observe:

- `[*]` — built-in (`y`)
- `[M]` — module (`m`)
- `[ ]` — disabled (`n`)
- `<*>`, `<M>`, `< >` — same but for sub-options
- Press `/` to search for a config symbol
- Press `?` for help on any option

Key sections to explore:

```
General setup --->
  Local version - append to kernel release
  Default hostname

Processor type and features --->
  Processor family
  Number of CPUs
  Preemption Model
```

```
Device Drivers --->
    (Browse the massive driver tree)

File systems --->
    (See all supported filesystems)

Kernel hacking --->
    (Debug options – very useful for development)
```

Step 3: Make targeted configuration changes:

```
# Search for a specific option in menuconfig
# Press / and type: EXT4
# Note: CONFIG_EXT4_FS

# Or use scripts directly
scripts/config --enable CONFIG_IKCONFIG
scripts/config --enable CONFIG_IKCONFIG_PROC
scripts/config --set-str CONFIG_LOCALVERSION "-mykernel"
scripts/config --disable CONFIG_DEBUG_INFO_BTFF

# Verify changes
grep "LOCALVERSION" .config
grep "IKCONFIG" .config
```

Step 4: Compare configurations:

```
# Compare with running kernel's config
diff <(grep "=y" .config | sort) \
    <(zcat /proc/config.gz 2>/dev/null | grep "=y" | sort) | head -30

# Or use the kernel's diff script
scripts/diffconfig /boot/config-$(uname -r) .config | head -30
```

✅ **Checkpoint:** You should have a `.config` file customized with your local version string.

🔗 Exercise 3.2: Building the Kernel

Objective: Compile a custom kernel from source.

⚠️ **Do this inside a VM!** A bad kernel can make your system unbootable.

Step 1: Build the kernel:

```
cd /tmp/linux

# Check how many CPUs available for parallel build
nproc

# Build the kernel image (use -j for parallel)
make -j$(nproc)

# This produces:
# - vmlinux          (uncompressed kernel, with symbols)
# - arch/x86/boot/bzImage (compressed bootable kernel)
```

```
ls -lh vmlinux
ls -lh arch/x86/boot/bzImage
```

Step 2: Build the modules:

```
make modules -j$(nproc)

# Count built modules
find . -name "*.ko" | wc -l
```

Step 3: Install (inside a VM only!):

```
# Install modules to /lib/modules/<version>/
sudo make modules_install

# Install the kernel
sudo make install

# This typically:
# 1. Copies bzImage to /boot/vmlinuz-<version>
# 2. Generates initramfs
# 3. Updates bootloader (GRUB)

# Verify
ls /boot/vmlinuz-*
ls /boot/initrd.img-* 2>/dev/null || ls /boot/initramfs-*

# Update GRUB
sudo update-grub
```

Step 4: Reboot and verify:

```
# Reboot into your new kernel
sudo reboot

# After reboot, verify
uname -r    # Should show your custom version
uname -v    # Should show your build timestamp
```

 **Challenge:** Build a minimal kernel that boots successfully. Disable as many features as possible while keeping the system functional. How small can you make the `bzImage` ?

🔗 Exercise 3.3: Kernel Build Artifacts

Objective: Understand what each build output file is.

Step 1: Examine build artifacts:

```
cd /tmp/linux

# vmlinux — the raw uncompressed kernel ELF
file vmlinux
size vmlinux
```

```
# bzImage – compressed, bootable kernel
file arch/x86/boot/bzImage
ls -lh arch/x86/boot/bzImage

# System.map – symbol-to-address mapping
head -20 System.map
wc -l System.map
grep " T start_kernel" System.map

# Module files
find . -name "*.ko" -type f | head -10
file $(find . -name "*.ko" -type f | head -1)
```

Step 2: Understand the relationship:

Source Code

```
↓ (make)
vmlinux (ELF, ~200MB, with debug symbols)
↓ (objcopy + compression)
bzImage (~10MB, stripped + compressed, bootable)
↓ (installed to /boot)
/boot/vmlinuz-<version>
```

```
# Compare sizes
ls -lh vmlinux arch/x86/boot/bzImage

# Strip symbols to see the difference
cp vmlinux vmlinux.stripped
strip vmlinux.stripped
ls -lh vmlinux vmlinux.stripped arch/x86/boot/bzImage
rm vmlinux.stripped
```

Step 3: Clean up build artifacts:

```
# Remove generated files but keep .config
make clean

# Remove everything including .config
# make mrproper

# Remove everything including editor backups
# make distclean
```

✓ **Checkpoint:** You understand vmlinux vs bzImage, System.map, and the build/install process.

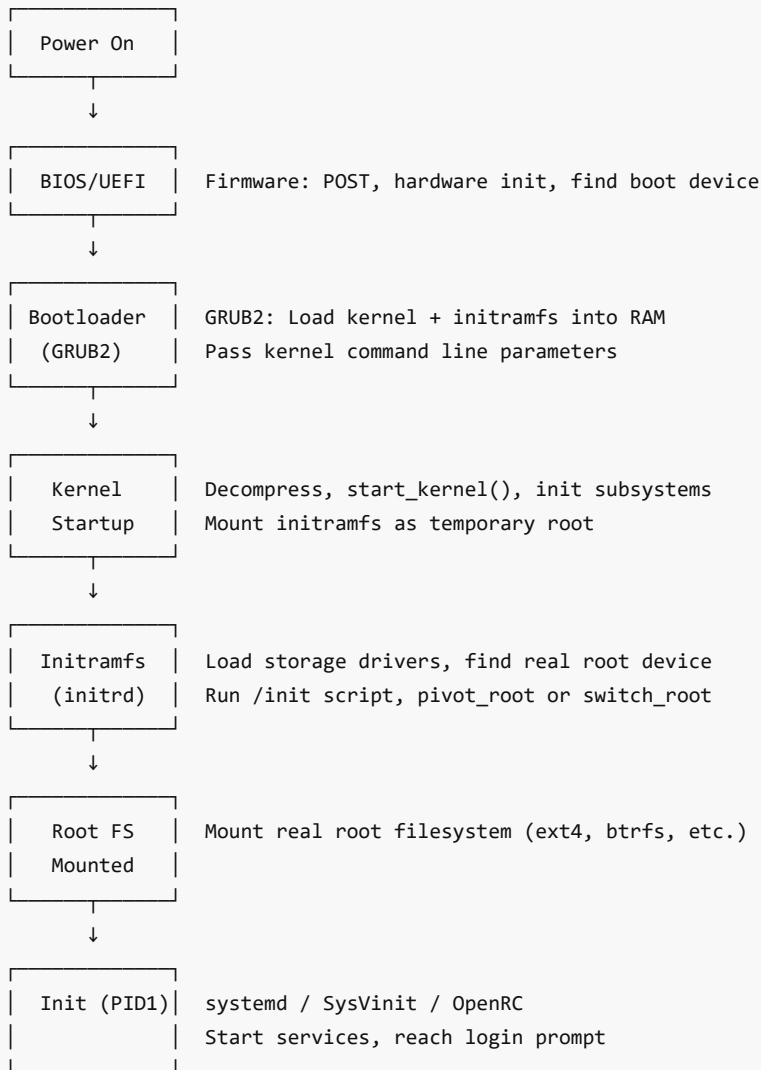
LAB 4: Boot Sequence, Rootfs & Initramfs

Mission: "Understanding the Boot Chain"

Story: A critical embedded system fails to boot after a firmware update. You need to understand every step of the Linux boot process to diagnose where it's failing — from power-on to the login prompt.

Background

Complete Linux Boot Sequence:



❏ Exercise 4.1: Tracing the Boot Process

Objective: Follow the boot sequence step by step on a real system.

Step 1: Examine GRUB configuration:

```
# GRUB config
cat /boot/grub/grub.cfg 2>/dev/null | grep -A 3 "menuentry" | head -20

# Default boot entry
grep "GRUB_DEFAULT" /etc/default/grub

# Kernel command line GRUB passes
grep "GRUB_CMDLINE" /etc/default/grub

# Currently booted kernel cmdline
cat /proc/cmdline
```

Step 2: Trace kernel boot messages:

```
# Full boot log (kernel ring buffer)
dmesg | head -60

# Key boot milestones
dmesg | grep -i "command line"
dmesg | grep -i "memory"
dmesg | grep -i "cpu"
dmesg | grep -i "mount"
dmesg | grep -i "init"

# Boot timing
dmesg | grep -E "^[.*)" | head -5      # Timestamps
systemd-analyze                        # Total boot time
systemd-analyze blame | head -15      # Slowest services
```

Step 3: Examine the bootloader files:

```
# What's in /boot?
ls -la /boot/

# Kernel images
ls -lh /boot/vmlinuz-*

# Initramfs images
ls -lh /boot/initrd.img-* 2>/dev/null || ls -lh /boot/initramfs-*

# System.map
ls -lh /boot/System.map-*

# Config
ls -lh /boot/config-*
```

Step 4: Answer in `answers.txt` :

1. What kernel command line parameters were passed at boot?
2. How long did your system take to boot (from kernel to login)?
3. What is the total size of your kernel + initramfs?

❏ Exercise 4.2: Root Filesystem (rootfs)

Objective: Understand what a root filesystem is and how to create one.

📖 Background

The **root filesystem (rootfs)** is the filesystem mounted at `/`. It contains everything the system needs to run:

<code>/</code>	← Root
<code>bin/</code>	← Essential binaries (ls, cp, mount)
<code>sbin/</code>	← System binaries (init, mount, ip)
<code>etc/</code>	← Configuration files
<code>lib/</code>	← Shared libraries
<code>dev/</code>	← Device nodes
<code>proc/</code>	← Process info (virtual)
<code>sys/</code>	← Sysfs (virtual)
<code>tmp/</code>	← Temporary files

var/	← Variable data (logs, spool)
usr/	← User programs and libraries
home/	← User home directories

Step 1: Examine your root filesystem:

```
# What filesystem type is /
mount | grep " / "
df -Th /

# Root filesystem size and usage
df -h /

# FHS – Filesystem Hierarchy Standard
ls -1 /
```

Step 2: Create a minimal rootfs with debootstrap:

```
# Create a minimal Debian rootfs
sudo mkdir -p /tmp/myroot
sudo debootstrap --variant=minbase bookworm /tmp/myroot http://deb.debian.org/debian

# Examine what was created
ls /tmp/myroot/
du -sh /tmp/myroot/

# Chroot into it
sudo chroot /tmp/myroot /bin/bash
# Inside chroot:
cat /etc/os-release
ls /
exit
```

Step 3: Create a truly minimal rootfs manually:

```
cd code/kernel_programming/lab04_boot/

# Run the script that builds a minimal rootfs
cat build_minimal_rootfs.sh
bash build_minimal_rootfs.sh

# Examine the result
ls -la /tmp/minimal_rootfs/
find /tmp/minimal_rootfs/ -type f
```

 **Challenge:** Create the smallest possible rootfs that can boot to a shell. Use busybox for a single-binary approach.

🔍 Exercise 4.3: Initramfs Deep Dive

Objective: Understand, inspect, and create initramfs images.

Background

Initramfs (initial RAM filesystem) is a temporary root filesystem loaded into RAM by the bootloader. It contains just enough to mount the real root filesystem:

```
Boot → Kernel starts → Initramfs mounted as / →  
→ Load storage drivers (SCSI, NVMe, RAID, LVM, crypto) →  
→ Find and mount real root device →  
→ switch_root to real rootfs →  
→ Run /sbin/init (systemd)
```

Why do we need it?

- Root filesystem might be on a device that needs drivers (RAID, LVM, encrypted)
- Those drivers are compiled as modules (not built into the kernel)
- Chicken-and-egg: can't load modules without a filesystem, can't mount filesystem without modules
- Initramfs solves this by providing a minimal filesystem with needed modules

Step 1: Inspect the existing initramfs:

```
# List initramfs contents  
lsinitramfs /boot/initrd.img-$(uname -r) 2>/dev/null | head -40  
  
# Or if lsinitramfs is not available  
mkdir -p /tmp/initramfs_inspect  
cd /tmp/initramfs_inspect  
  
# Initramfs is a compressed cpio archive  
# Detect compression type  
file /boot/initrd.img-$(uname -r) 2>/dev/null || \  
    file /boot/initramfs-$(uname -r).img  
  
# Extract it (may be gzip, lz4, zstd, or microcode + cpio)  
unmkinitramfs /boot/initrd.img-$(uname -r) /tmp/initramfs_inspect 2>/dev/null  
  
# Examine contents  
ls /tmp/initramfs_inspect/  
find /tmp/initramfs_inspect/ -maxdepth 2 -type f | head -30
```

Step 2: Understand initramfs structure:

```
# Key files inside initramfs  
find /tmp/initramfs_inspect/ -name "init" -type f 2>/dev/null  
find /tmp/initramfs_inspect/ -name "*.ko*" | head -10    # Kernel modules  
  
# The init script – this runs first  
cat /tmp/initramfs_inspect/main/init 2>/dev/null | head -40
```

Step 3: Create a custom initramfs:

```
cd code/kernel_programming/lab04_boot/  
cat build_initramfs.sh  
bash build_initramfs.sh  
  
# This creates a minimal initramfs with:  
# - busybox (all basic commands)  
# - /init script  
# - /dev, /proc, /sys mount points
```

Step 4: Regenerate the system initramfs:


```
# Update initramfs for current kernel
sudo update-initramfs -u -k $(uname -r)

# Or create a new one
sudo update-initramfs -c -k $(uname -r)

# Verify
ls -lh /boot/initrd.img-$(uname -r)
```

✔ **Checkpoint:** You can inspect and create initramfs images.

🔗 Exercise 4.4: Startup Init Program

Objective: Understand the init system — the first userspace process (PID 1).

📖 Background

After the kernel mounts rootfs, it executes the **init program** — process ID 1. This process is responsible for starting all other services.

Init systems (historical to modern):

Init System	Era	Config Location	Key Feature
SysVinit	1983	/etc/inittab, /etc/init.d/	Sequential, runlevels
Upstart	2006	/etc/init/	Event-driven, parallel
systemd	2010	/etc/systemd/, /lib/systemd/	Modern, parallel, socket/dbus activation
OpenRC	2007	/etc/conf.d/	Gentoo, dependency-based

Step 1: Identify your init system:

```
# What is PID 1?
ps -p 1 -o comm=
ls -la /sbin/init

# For systemd:
systemctl --version
systemd-analyze

# Boot target (equivalent of runlevel)
systemctl get-default
```

Step 2: Explore systemd (if applicable):

```
# List all units
systemctl list-units --type=service | head -20

# List failed units
systemctl list-units --failed

# Service dependencies
systemctl list-dependencies default.target | head -20

# Boot timeline
```

```
systemd-analyze critical-chain | head -15

# PID 1 status
systemctl status
```

Step 3: Examine the init sequence:

```
# Kernel init parameter
cat /proc/cmdline | grep -o "init=[^ ]*"

# What the kernel searches for by default:
# 1. /sbin/init
# 2. /etc/init
# 3. /bin/init
# 4. /bin/sh (fallback)

# Kernel's start_kernel() → rest_init() → kernel_init() → run_init_process()
```

Step 4: Create a custom init program:

```
cd code/kernel_programming/lab04_boot/
cat custom_init.c
make custom_init

# This is a minimal init that:
# 1. Mounts /proc and /sys
# 2. Prints system info
# 3. Launches a shell
```

 **Challenge:** Boot a QEMU VM with your custom init program instead of systemd. Modify the kernel command line to use `init=/custom_init`.

LAB 5: Storage Strategy & Virtual Kernels

Mission: "The Virtual Testing Ground"

Story: Your team needs to test kernel changes rapidly without risking hardware. You must set up virtual kernel environments — using both UML (User-Mode Linux) and QEMU — to create a safe sandbox for kernel experimentation.

Background

Storage Strategies for Embedded/IoT:

Strategy	Description	Use Case
NOR Flash	Direct-execute, small, expensive	Bootloader, firmware
NAND Flash	Large, cheap, needs FTL	Root filesystem (UBIFS, JFFS2)
eMMC	NAND + controller, block device	Android, embedded Linux
SD Card	Removable, standard block device	Raspberry Pi, dev boards
NFS Root	Network-mounted root filesystem	Development, diskless boot

RAM disk	Root in memory (initramfs only)	Recovery, installer
Overlay FS	Read-only base + writable overlay	Docker, Live USBs

❏ Exercise 5.1: Storage Strategies

Objective: Understand different root filesystem storage approaches.

Step 1: Examine your current storage:

```
# Block devices and their filesystems
lsblk -f

# Mount points
mount | grep -v "tmpfs\|cgroup\|sys\|proc"

# Disk usage
df -Th | grep -v "tmpfs\|udev"
```

Step 2: Create and use different storage types:

```
# Create a loop device (simulates block storage)
dd if=/dev/zero of=/tmp/disk.img bs=1M count=64
mkfs.ext4 /tmp/disk.img
mkdir -p /tmp/loopmount
sudo mount -o loop /tmp/disk.img /tmp/loopmount
echo "Hello from loop device!" | sudo tee /tmp/loopmount/hello.txt
ls -la /tmp/loopmount/
sudo umount /tmp/loopmount

# Create a RAM disk (tmpfs)
sudo mount -t tmpfs -o size=32M tmpfs /tmp/ramdisk 2>/dev/null || \
    mkdir -p /tmp/ramdisk
echo "This lives in RAM!" > /tmp/ramdisk/test.txt
df -h /tmp/ramdisk

# Create an overlay filesystem
mkdir -p /tmp/overlay/{lower,upper,work,merged}
echo "base file" > /tmp/overlay/lower/base.txt
sudo mount -t overlay overlay \
    -o lowerdir=/tmp/overlay/lower,upperdir=/tmp/overlay/upper,workdir=/tmp/overlay/work \
    /tmp/overlay/merged
cat /tmp/overlay/merged/base.txt
echo "new content" > /tmp/overlay/merged/overlay.txt
ls /tmp/overlay/upper/      # Only the new/changed files
sudo umount /tmp/overlay/merged
```

Step 3: NFS root setup (for development):

```
# Install NFS server (on host)
sudo apt install -y nfs-kernel-server

# Export a directory
echo "/tmp/myroot *(rw, sync, no_subtree_check, no_root_squash)" | \
    sudo tee -a /etc/exports
```

```
sudo exportfs -ra

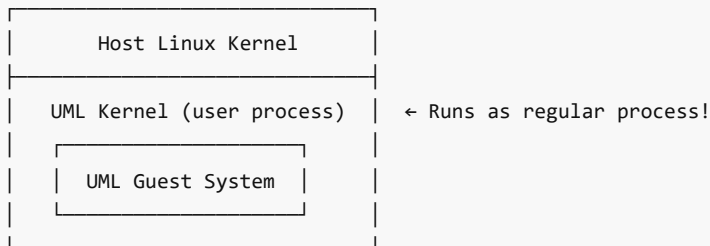
# Kernel command line for NFS root:
# root=/dev/nfs nfsroot=192.168.1.1:/tmp/myroot ip=dhcp
```

❧ Exercise 5.2: UML Kernel (User-Mode Linux)

Objective: Build and run a Linux kernel as a userspace process.

📖 Background

User-Mode Linux (UML) compiles the kernel as a regular userspace program. It runs as a process on your host, making it perfect for kernel development and testing without VMs or reboots.



Step 1: Build a UML kernel:

```
cd /tmp/linux # Your kernel source

# Configure for UML
make ARCH=um defconfig

# Enable useful options
make ARCH=um menuconfig
# → UML-specific options → Enable host filesystem
# → Enable networking

# Build
make ARCH=um -j$(nproc)

# The result is a regular ELF executable!
file linux
ls -lh linux
```

Step 2: Create a rootfs for UML:

```
# Create an ext4 disk image for UML
dd if=/dev/zero of=/tmp/uml_root.img bs=1M count=512
mkfs.ext4 /tmp/uml_root.img

# Populate it
mkdir -p /tmp/uml_mount
sudo mount -o loop /tmp/uml_root.img /tmp/uml_mount
sudo debootstrap --variant=minbase bookworm /tmp/uml_mount http://deb.debian.org/debian

# Set root password
sudo chroot /tmp/uml_mount passwd root
```

```
sudo umount /tmp/uml_mount
```

Step 3: Run UML:

```
./linux ubda=/tmp/uml_root.img mem=256M

# You should get a login prompt
# Login as root with the password you set

# Inside UML:
uname -a          # Should show "um" architecture
cat /proc/cpuinfo
ls /
```

Step 4: Debug the UML kernel with GDB:

```
# UML is just a process – you can GDB it!
gdb ./linux
(gdb) break start_kernel
(gdb) run ubda=/tmp/uml_root.img mem=256M
```

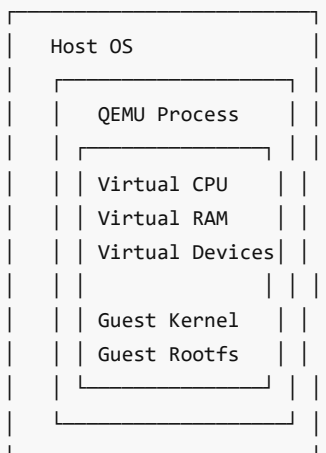
 **Challenge:** Make UML networking work so the guest can access the internet. Use the `eth0=slirp` option.

❏ Exercise 5.3: QEMU Kernel

Objective: Boot a custom kernel using QEMU — the standard kernel developer workflow.

Background

QEMU is a full-system emulator. It's the primary tool kernel developers use for testing:



Step 1: Boot your custom kernel with QEMU:

```
cd /tmp/linux

# Make sure you've built the kernel
ls arch/x86/boot/bzImage

# Create a simple initramfs to test booting
```

```

cd code/kernel_programming/lab05_virtual/
bash build_qemu_initramfs.sh    # Creates /tmp/test_initramfs.cpio.gz

# Boot with QEMU
qemu-system-x86_64 \
    -kernel /tmp/linux/arch/x86/boot/bzImage \
    -initrd /tmp/test_initramfs.cpio.gz \
    -append "console=ttyS0 rdinit=/init" \
    -nographic \
    -m 256M

# Press Ctrl+A then X to exit QEMU

```

Step 2: Boot with a full disk image:

```

# Create a QEMU disk image
qemu-img create -f qcow2 /tmp/qemu_disk.qcow2 4G

# Install a minimal system
sudo debootstrap --variant=minbase bookworm /tmp/qemu_rootfs http://deb.debian.org/debian

# Or use the disk image approach
qemu-system-x86_64 \
    -kernel /tmp/linux/arch/x86/boot/bzImage \
    -hda /tmp/uml_root.img \
    -append "root=/dev/sda console=ttyS0" \
    -nographic \
    -m 512M

```

Step 3: QEMU with debugging support:

```

# Start QEMU with GDB server
qemu-system-x86_64 \
    -kernel /tmp/linux/arch/x86/boot/bzImage \
    -initrd /tmp/test_initramfs.cpio.gz \
    -append "console=ttyS0 rdinit=/init nokaslr" \
    -nographic \
    -m 256M \
    -s -S    # -s: GDB on port 1234, -S: pause at start

# In another terminal, connect GDB
gdb /tmp/linux/vmlinux
(gdb) target remote :1234
(gdb) break start_kernel
(gdb) continue

```

Step 4: QEMU with virtio devices (fast I/O):

```

qemu-system-x86_64 \
    -kernel /tmp/linux/arch/x86/boot/bzImage \
    -initrd /tmp/test_initramfs.cpio.gz \
    -append "console=ttyS0 rdinit=/init" \
    -nographic \
    -m 512M \
    -netdev user,id=net0 -device virtio-net-pci,netdev=net0 \

```

```
-drive file=/tmp/qemu_disk.qcow2,if=virtio \
-enable-kvm      # Hardware acceleration (if available)
```

 **Challenge:** Set up a QEMU-based kernel development workflow: edit kernel source → build → boot in QEMU → test → iterate. Create a script that automates this cycle.

 **Checkpoint:** You can boot custom kernels in both UML and QEMU environments.

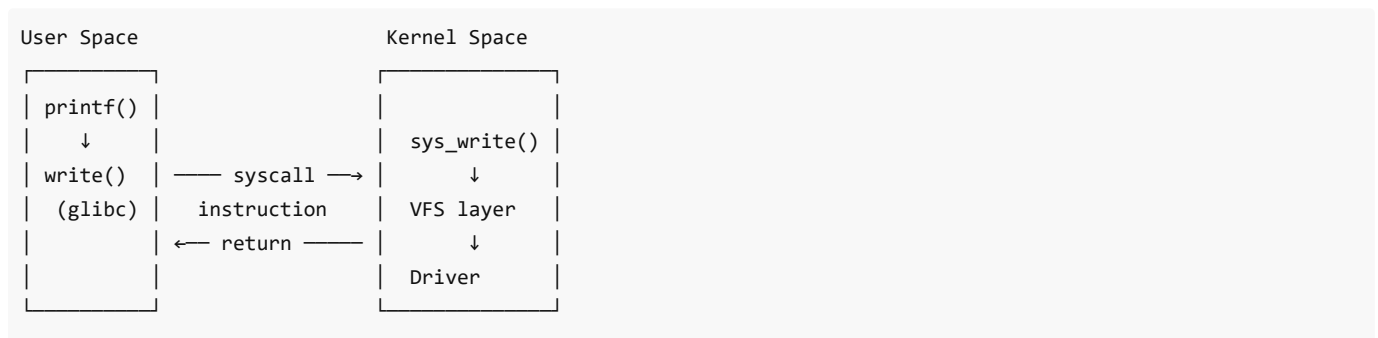
LAB 6: System Calls

Mission: "The Gateway Between Worlds"

Story: You've been tasked with understanding the exact mechanism that allows userspace programs to request kernel services. You'll trace existing syscalls, understand the calling convention, and even add your own.

Background

System calls are the controlled entry points from user space into kernel space.



x86_64 Linux syscall convention:

Register	Purpose
<code>rax</code>	Syscall number
<code>rdi</code>	Argument 1
<code>rsi</code>	Argument 2
<code>rdx</code>	Argument 3
<code>r10</code>	Argument 4
<code>r8</code>	Argument 5
<code>r9</code>	Argument 6
<code>rax</code>	Return value

🔧 Exercise 6.1: Tracing System Calls

Objective: Observe and understand system calls made by programs.

Step 1: Trace system calls with `strace`:

```
# Trace all syscalls of a command
strace ls /tmp 2>&1 | head -30
```

```
# Count syscalls by type
strace -c ls /tmp 2>&1

# Trace specific syscalls
strace -e trace=open,read,write cat /etc/hostname 2>&1

# Trace only file-related syscalls
strace -e trace=file ls /tmp 2>&1

# Trace only network syscalls
strace -e trace=network curl -s http://localhost 2>&1 | head -20

# Follow child processes
strace -f bash -c "echo hello | cat" 2>&1 | tail -20
```

Step 2: Examine the syscall table:

```
# Syscall numbers
grep -r "define __NR_" /usr/include/asm-generic/unistd.h 2>/dev/null | head -20

# Or from the kernel source
grep -r "define __NR_" /tmp/linux/arch/x86/include/generated/uapi/asm/unistd_64.h 2>/dev/null | head -30

# Or use ausyscall
ausyscall --dump 2>/dev/null | head -30
```

Step 3: Navigate to the lab directory:

```
cd code/kernel_programming/lab06_syscalls/
cat syscall_trace.c
```

Step 4: Build and trace:

```
make
strace -c ./syscall_trace 2>&1
```

❏ Exercise 6.2: Making Raw System Calls

Objective: Invoke system calls directly without using the C library.

Step 1: Examine the raw syscall program:

```
cat raw_syscall.c
```

Step 2: Compile and run:

```
make raw_syscall
./raw_syscall
```

This demonstrates:

- Using `syscall()` to bypass glibc wrappers
- Direct assembly `syscall` instruction

- Understanding the registers used

Step 3: Compare library call vs raw syscall:

```
# Trace both
strace ./syscall_trace 2>&1 | grep write | head -5
strace ./raw_syscall 2>&1 | grep write | head -5
```

❏ Exercise 6.3: Adding a Custom System Call

Objective: Add a new system call to the Linux kernel.

⚠ **This requires kernel source and rebuild. Use QEMU!**

Step 1: Examine the template:

```
cat custom_syscall_patch.txt
```

Step 2: Apply changes to kernel source (following the template):

```
cd /tmp/linux

# 1. Add syscall number to the table
# File: arch/x86/entry/syscalls/syscall_64.tbl
# Add at the end:
# 451 common hello sys_hello

# 2. Add the function prototype
# File: include/linux/syscalls.h
# Add: asmlinkage long sys_hello(const char __user *name, size_t len);

# 3. Implement the syscall
# Create: kernel/sys_hello.c
cat > kernel/sys_hello.c << 'SYSCALL_EOF'
#include <linux/kernel.h>
#include <linux/syscalls.h>
#include <linux/uaccess.h>

SYSCALL_DEFINE2(hello, const char __user *, name, size_t, len)
{
    char buf[64];

    if (len > sizeof(buf) - 1)
        return -EINVAL;

    if (copy_from_user(buf, name, len))
        return -EFAULT;

    buf[len] = '\0';
    pr_info("Hello from kernel, %s!\n", buf);

    return 0;
}
SYSCALL_EOF

# 4. Add to Makefile
```

```
# File: kernel/Makefile
# Add sys_hello.o to obj-y

# 5. Rebuild
make -j$(nproc)
```

Step 3: Test the custom syscall:

```
# Write a test program
cat > /tmp/test_hello_syscall.c << 'EOF'
#include <stdio.h>
#include <unistd.h>
#include <sys/syscall.h>
#include <string.h>

#define __NR_hello 451

int main(void) {
    const char *name = "Kernel Developer";
    long ret = syscall(__NR_hello, name, strlen(name));

    if (ret == 0)
        printf("Syscall succeeded!\n");
    else
        perror("Syscall failed");

    return ret;
}
EOF

gcc -o /tmp/test_hello_syscall /tmp/test_hello_syscall.c

# Boot in QEMU with the new kernel and run the test
# Then check dmesg for the "Hello from kernel" message
```

 **Challenge:** Add a system call `sys_uptime_ms` that returns the system uptime in milliseconds. Test it from user space.

 **Checkpoint:** You understand how syscalls work from user space to kernel space, and can add new ones.



Mini-Project 1: Kernel Info Dashboard

Mission: "System Intelligence Report"

Build a comprehensive system information tool that queries kernel information through multiple interfaces (/proc, /sys, syscalls) and presents a formatted dashboard.

Requirements

1. System Overview Panel

- Kernel version, hostname, architecture
- Uptime, load averages
- CPU count and model
- Total/available memory

2. Boot Information Panel

- Boot command line
- Init system type and version
- Boot time and date
- Rootfs type and size

3. Kernel Module Panel

- o Total loaded modules count
- o Top 10 largest modules (by size)
- o Module dependency tree for a specified module

4. Device Summary Panel

- Character device count and majors
- Block device count with sizes
- Network interface list with status

Deliverables

```
code/kernel_programming/mini_project1_kernel_dashboard/
├── dashboard.c           # Main dashboard program
├── sysinfo_collector.c   # /proc and /sys reader
├── sysinfo_collector.h   # Header
├── Makefile
└── README.md
```

Implementation Hints

```
/* Read from /proc */
FILE *fp = fopen("/proc/version", "r");
fgets(buf, sizeof(buf), fp);

/* Read from /sys */
fp = fopen("/sys/class/block/sda/size", "r");

/* Use syscalls directly */
struct sysinfo si;
sysinfo(&si); /* uptime, loads, memory */

/* uname syscall */
struct utsname uts;
uname(&uts); /* kernel version, hostname, arch */
```

Expected Output

Kernel Information Dashboard	
Kernel:	6.8.0-mykernel
Hostname:	dev-machine
Arch:	x86_64
Uptime:	2d 5h 32m
Load:	0.45, 0.32, 0.28
CPUs:	4 x AMD Ryzen 5

```
|| Memory:    15.6 GiB total, 8.2 GiB available ||
||-----||
|| Boot:      root=/dev/sda1 console=tty0      ||
|| Init:      systemd v252                    ||
|| RootFS:    ext4 on /dev/sda1 (47.2 GiB)     ||
||-----||
|| Modules:   142 loaded                      ||
|| Devices:   23 char, 8 block, 3 network      ||
||-----||
```



Mini-Project 2: QEMU Kernel Test Harness

Mission: "The Automated Kernel Lab"

Build an automated system that builds a kernel, boots it in QEMU, runs tests inside the guest, and collects results — the foundation of kernel CI/CD.

Requirements

1. Build Script

- Accept kernel source path and config options
- Build kernel and modules with parallel make
- Generate a minimal initramfs with test programs
- Report build time and binary sizes

2. Boot Script

- Launch QEMU with the built kernel
- Use serial console for output capture
- Set a boot timeout (panic if stuck)
- Support both initramfs-only and disk-based boots

3. Test Runner

- Run inside the QEMU guest via initramfs `/init`
- Execute a series of kernel tests
- Print pass/fail results in a parseable format
- Poweroff after tests complete

4. Result Collector

- Parse QEMU serial output for test results
- Generate a summary report
- Detect kernel panics or hangs

Deliverables

```
code/kernel_programming/mini_project2_qemu_harness/
├─ build_kernel.sh      # Kernel build automation
├─ create_initramfs.sh  # Initramfs builder with tests
├─ run_qemu.sh          # QEMU launcher
├─ test_runner.c        # Test program (runs inside guest)
├─ parse_results.sh     # Output parser
├─ Makefile
└─ README.md
```

Test Runner Format

```
[TEST] kernel_version ... PASS
[TEST] proc_filesystem ... PASS
[TEST] sysfs_mounted ... PASS
[TEST] memory_available ... PASS (15842 MB)
[TEST] module_load ... FAIL (modprobe not found)
=== RESULTS: 4 passed, 1 failed ===
```



Capstone Project: Custom Bootable Linux System

Mission: "From Source to Boot — Build Your Own Linux"

Construct a complete, bootable Linux system from scratch. You'll build the kernel, create a rootfs, set up a bootloader, and boot it in QEMU. This is the ultimate test of your Part 1 knowledge.

Phase 1: Kernel Build

- Download and configure kernel source (minimal config)
- Build with custom `CONFIG_LOCALVERSION="-capstone"`
- Target: `bzImage` under 8 MB

Phase 2: Root Filesystem

- Create minimal rootfs with busybox (statically linked)
- Include: `/bin`, `/sbin`, `/etc`, `/proc`, `/sys`, `/dev`, `/tmp`
- Create `/etc/inittab` and init scripts
- Add at least one custom program

Phase 3: Initramfs

- Build initramfs from the rootfs
- Write `/init` script that:
 - Mounts `/proc` and `/sys`
 - Creates device nodes
 - Detects and mounts the root device
 - Performs `switch_root`

Phase 4: Boot in QEMU

- Create a bootable disk image with GRUB
- Install kernel and initramfs
- Boot and verify system works
- Demonstrate: login, run programs, view `dmesg`

Phase 5: System Call Integration

- Add one custom syscall to the kernel
- Build a userspace tool that uses it
- Include the tool in the rootfs
- Demonstrate the custom syscall working

Deliverables

```
code/kernel_programming/capstone/
├── scripts/
│   └── build_kernel.sh
```

```

|   ├── build_rootfs.sh
|   ├── build_initramfs.sh
|   ├── create_disk.sh
|   └── run_qemu.sh
├── rootfs_overlay/          # Custom files for rootfs
|   ├── etc/
|   |   ├── inittab
|   |   ├── init.d/rcS
|   |   └── hostname
|   └── usr/local/bin/
|       └── sysinfo          # Custom program
├── kernel_patches/
|   └── custom_syscall.patch # Your custom syscall
├── Makefile                 # Orchestrates everything
└── README.md

```

Evaluation Criteria

- System boots to a shell in QEMU
- Kernel has custom version string
- Root filesystem is under 16 MB
- Custom syscall works end-to-end
- Init sequence is clean (no errors in dmesg)
- Documentation explains each step

LAB 7: Kernel Helper Functions & Kernel Logs 🍓

🎯 Mission: "The Kernel's Toolbox"

Story: Now that you can build and boot kernels, it's time to write actual kernel code. But the kernel has no libc — no `printf()`, no `malloc()`, no `strlen()`. You must learn the kernel's own helper functions and its logging system before writing any driver code.

📖 Background

The kernel provides its own versions of common functions. You **cannot** use standard C library functions in kernel code.

User Space (libc)	Kernel Space	Header
<code>printf()</code>	<code>printk()</code>	<code><linux/kernel.h></code>
<code>malloc()</code> / <code>free()</code>	<code>kmalloc()</code> / <code>kfree()</code>	<code><linux/slab.h></code>
<code>memcpy()</code>	<code>memcpy()</code>	<code><linux/string.h></code>
<code>strcmp()</code>	<code>strcmp()</code>	<code><linux/string.h></code>
<code>snprintf()</code>	<code>snprintf()</code>	<code><linux/kernel.h></code>
<code>sleep()</code>	<code>msleep()</code> , <code>ssleep()</code>	<code><linux/delay.h></code>
<code>errno</code>	Return negative error codes	<code><linux/errno.h></code>
<code>assert()</code>	<code>BUG_ON()</code> , <code>WARN_ON()</code>	<code><linux/bug.h></code>

Kernel string/memory helpers:

```
#include <linux/string.h>
/* String functions (same names, kernel versions) */
strlen(), strcmp(), strncmp(), strcpy(), strncpy()
strcat(), strncat(), strchr(), strrchr(), strstr()
memcpy(), memset(), memcmp(), memmove()

#include <linux/kernel.h>
/* Formatted printing */
snprintf(), scnprintf(), sscanf()
simple_strtol(), simple_strtoul()
kstrtoint(), kstrtol(), kstrtou32() /* preferred over simple_strto* */

#include <linux/list.h>
/* Linked lists – kernel's most-used data structure */
INIT_LIST_HEAD(), list_add(), list_del(), list_for_each_entry()

#include <linux/math.h>
/* Math helpers */
DIV_ROUND_UP(), min(), max(), clamp(), abs()
```

❏ Exercise 7.1: Printk and Kernel Log Levels

Objective: Understand the kernel logging system.

Printk Log Levels

```
/* Log levels – defined in <linux/kern_levels.h> */
pr_emerg("...") /* KERN_EMERG - 0 – System is unusable */
pr_alert("...") /* KERN_ALERT - 1 – Action must be taken immediately */
pr_crit("...") /* KERN_CRIT - 2 – Critical conditions */
pr_err("...") /* KERN_ERR - 3 – Error conditions */
pr_warn("...") /* KERN_WARNING - 4 – Warning conditions */
pr_notice("...") /* KERN_NOTICE - 5 – Normal but significant */
pr_info("...") /* KERN_INFO - 6 – Informational */
pr_debug("...") /* KERN_DEBUG - 7 – Debug (needs CONFIG_DYNAMIC_DEBUG) */
```

Step 1: Examine current kernel log settings:

```
# Current console log level
cat /proc/sys/kernel/printk
# Format: current default minimum boot-default

# Meaning:
# 4    4    1    7
# |    |    |    └─ boot-default level
# |    |    └─ minimum level (most important)
# |    └─ default level for printk without level
# └─ current console level (messages <= this are shown)

# Temporarily change console level (show all messages)
sudo bash -c "echo 8 > /proc/sys/kernel/printk"

# View kernel log
```

```
dmesg | tail -20
dmesg -w # Follow mode (like tail -f)
```

Step 2: Navigate to the lab and examine the module:

```
cd code/kernel_programming/lab07_helpers/
cat printk_demo.c
```

Step 3: Build and test:

```
make
sudo insmod printk_demo.ko
dmesg | tail -20

# See the different log levels in action
sudo rmmod printk_demo
dmesg | tail -10
```

 **Raspberry Pi:** This works identically on RPi. Use `sudo apt install raspberrypi-kernel-headers` for headers.

Step 4: Explore dynamic debug:

```
# List available debug messages
sudo cat /sys/kernel/debug/dynamic_debug/control | head -20

# Enable debug messages from a specific module
sudo bash -c "echo 'module printk_demo +p' > /sys/kernel/debug/dynamic_debug/control"

# Enable debug messages from a specific file
sudo bash -c "echo 'file printk_demo.c +p' > /sys/kernel/debug/dynamic_debug/control"
```

Step 5: Rate-limited printing:

```
# View rate-limited demo in the module
# printk_ratelimited() prevents log flooding
```

 **Challenge:** Write a module that uses `pr_info()` with format specifiers to print: pointer values (`%p` vs `%px`), kernel addresses (`%pK`), and MAC addresses (`%pM`). Observe what the kernel does with the `%p` specifier for security.

🔗 Exercise 7.2: Kernel Helper Functions in Action

Objective: Use kernel-specific helper functions in a module.

Step 1: Examine the helper demo module:

```
cat helper_functions.c
```

Step 2: Build, load, and observe:

```
make
sudo insmod helper_functions.ko
dmesg | tail -40
```



```
sudo rmmod helper_functions
dmesg | tail -5
```

This module demonstrates:

- `kstrtoint()` , `kstrtoul()` — safe string-to-integer conversion
- `snprintf()` — formatted string building
- `memset()` , `memcpy()` , `memcmp()` — memory operations
- `min()` , `max()` , `clamp()` — math helpers
- `IS_ERR()` , `PTR_ERR()` , `ERR_PTR()` — error pointer idiom
- `jiffies` , `msecs_to_jiffies()` — time helpers
- `BUG_ON()` , `WARN_ON()` — assertion macros

✅ **Checkpoint:** You understand that the kernel has its own function library, distinct from libc.

LAB 8: Module Programming 🍓

🎯 Mission: "Your First Kernel Module"

Story: Modules are the building blocks of Linux drivers. They allow you to add code to the running kernel without rebooting. Every driver starts as a module. Master this, and you can write any driver.

📖 Background

A kernel module is a piece of code that can be loaded/unloaded at runtime:

```
#include <linux/module.h>
#include <linux/init.h>

static int __init mymodule_init(void) {
    pr_info("Module loaded\n");
    return 0; /* 0 = success, negative = error */
}

static void __exit mymodule_exit(void) {
    pr_info("Module unloaded\n");
}

module_init(mymodule_init);
module_exit(mymodule_exit);

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Your Name");
MODULE_DESCRIPTION("My first kernel module");
MODULE_VERSION("1.0");
```

Module commands:

<code>insmod module.ko</code>	# Load module (no dependency resolution)
<code>modprobe module</code>	# Load module + dependencies
<code>rmmod module</code>	# Unload module
<code>lsmod</code>	# List loaded modules
<code>modinfo module.ko</code>	# Show module information

❧ Exercise 8.1: Hello World Module

Objective: Write, build, load, and unload your first kernel module.

Step 1: Navigate and examine:

```
cd code/kernel_programming/lab08_modules/  
cat hello_module.c  
cat Makefile
```

Step 2: Build the module:

```
make  
  
# Examine the output  
file hello_module.ko  
modinfo hello_module.ko
```

🍷 **Raspberry Pi Makefile:** The same Makefile works on RPi — just ensure `raspberrypi-kernel-headers` is installed. The `KDIR` path auto-detects via `uname -r`.

Step 3: Load, verify, unload:

```
# Load  
sudo insmod hello_module.ko  
dmesg | tail -5  
lsmod | grep hello  
  
# Module info from sysfs  
ls /sys/module/hello_module/  
cat /sys/module/hello_module/version 2>/dev/null  
  
# Unload  
sudo rmmod hello_module  
dmesg | tail -5  
lsmod | grep hello # Should be empty
```

Step 4: Module with parameters:

```
cat param_module.c  
  
# Build and load with parameters  
make  
sudo insmod param_module.ko myint=42 mystr="hello" myarr=1,2,3  
  
# View parameters  
cat /sys/module/param_module/parameters/myint  
cat /sys/module/param_module/parameters/mystr  
  
# Change parameter at runtime (if permissions allow)  
sudo bash -c "echo 100 > /sys/module/param_module/parameters/myint"  
cat /sys/module/param_module/parameters/myint  
dmesg | tail -5
```

```
sudo rmmod param_module
```

❏ Exercise 8.2: Export Symbols

Objective: Understand how modules can share functions with other modules.

📖 Background

```
/* Module A exports a function */
int my_helper(int x) { return x * 2; }
EXPORT_SYMBOL(my_helper);          /* Available to all modules */
EXPORT_SYMBOL_GPL(my_helper);      /* Only to GPL-licensed modules */

/* Module B uses it */
extern int my_helper(int x);
```

Step 1: Examine the symbol export demo:

```
cat symbol_export.c    # Module that exports symbols
cat symbol_import.c    # Module that uses exported symbols
```

Step 2: Build and test:

```
make

# Load exporter first (order matters!)
sudo insmod symbol_export.ko
dmesg | tail -5

# Check exported symbols
grep "my_" /proc/kallsyms

# Load importer
sudo insmod symbol_import.ko
dmesg | tail -5

# Verify dependency
lsmod | grep symbol

# Unload in reverse order
sudo rmmod symbol_import
sudo rmmod symbol_export
dmesg | tail -10
```

Step 3: Explore the kernel symbol table:

```
# All exported symbols
sudo cat /proc/kallsyms | wc -l

# Find printk
sudo cat /proc/kallsyms | grep " printk$"
```

```
# Module symbols
sudo cat /proc/kallsyms | grep "\[symbol_export\]"

# Symbol types: T=text(code), D=data, B=BSS, R=readonly
sudo cat /proc/kallsyms | head -20
```

 **Challenge:** Create three modules: `math_ops.ko` (exports add, subtract, multiply), `string_ops.ko` (exports reverse, uppercase), and `test_ops.ko` (imports and tests both). Load/unload them correctly.

LAB 9: Procfs, Sysfs, Debugfs & Seqfile

Mission: "Windows to the Kernel"

Story: Kernel modules need to communicate with userspace. You'll learn the three main virtual filesystems that provide this interface, plus the seqfile API for handling large outputs efficiently.

Background

Interface	Mount Point	Purpose	Typical Use
procfs	<code>/proc</code>	Process and kernel info	Legacy driver config, stats
sysfs	<code>/sys</code>	Device/driver hierarchy	Modern driver attributes
debugfs	<code>/sys/kernel/debug</code>	Debug information	Development, tracing

🔗 Exercise 9.1: Procfs — Creating /proc Entries

Objective: Create a kernel module that exposes data via `/proc`.

Step 1: Examine the procfs demo:

```
cd code/kernel_programming/lab09_virtual_fs/
cat procfs_demo.c
```

Step 2: Build, load, and test:

```
make procfs_demo
sudo insmod procfs_demo.ko

# Read from /proc
cat /proc/mydriver_info
cat /proc/mydriver_stats

# Write to /proc
echo "hello" | sudo tee /proc/mydriver_config
cat /proc/mydriver_config

# List your proc entries
ls -la /proc/mydriver_*

sudo rmmod procfs_demo
dmesg | tail -10
```

 **Raspberry Pi:** Works identically. Great for reading RPi hardware info from kernel space.

❏ Exercise 9.2: Sysfs — The Modern Device Interface

Objective: Create sysfs attributes for a kernel module.

Step 1: Examine the sysfs demo:

```
cat sysfs_demo.c
```

Step 2: Build and test:

```
make sysfs_demo
sudo insmod sysfs_demo.ko

# Your sysfs entries appear under /sys/kernel/
ls /sys/kernel/mydevice/
cat /sys/kernel/mydevice/status
cat /sys/kernel/mydevice/value

# Write a value
echo 42 | sudo tee /sys/kernel/mydevice/value
cat /sys/kernel/mydevice/value

sudo rmmod sysfs_demo
```

❏ Exercise 9.3: Debugfs — Developer's Best Friend

Objective: Use debugfs for development and debugging.

Step 1: Examine the debugfs demo:

```
cat debugfs_demo.c
```

Step 2: Build and test:

```
make debugfs_demo
sudo insmod debugfs_demo.ko

# Mount debugfs if not already mounted
sudo mount -t debugfs none /sys/kernel/debug 2>/dev/null

# Access debug entries
ls /sys/kernel/debug/mydriver_debug/
cat /sys/kernel/debug/mydriver_debug/counter
cat /sys/kernel/debug/mydriver_debug/blob_data

# debugfs helpers for common types
cat /sys/kernel/debug/mydriver_debug/enable

sudo rmmod debugfs_demo
```

❏ Exercise 9.4: Seqfile — Handling Large Outputs

Objective: Use seq_file API for paginated /proc output.

📖 Background

The `seq_file` API solves the problem of outputting data larger than a single page (4 KB). It handles buffering, pagination, and seeking automatically.

```
/* seq_file operations */
static void *my_seq_start(struct seq_file *s, loff_t *pos);
static void *my_seq_next(struct seq_file *s, void *v, loff_t *pos);
static void my_seq_stop(struct seq_file *s, void *v);
static int my_seq_show(struct seq_file *s, void *v);
```

Step 1: Examine and test the seqfile demo:

```
cat seqfile_demo.c
make seqfile_demo
sudo insmod seqfile_demo.ko

# Read large output – seq_file handles pagination
cat /proc/seqfile_demo

# Read with offset
dd if=/proc/seqfile_demo bs=64 skip=1 count=1 2>/dev/null

sudo rmmod seqfile_demo
```

🎮 **Challenge:** Create a module that exposes all loaded modules and their sizes via `/proc/my_modules` using `seq_file`. Support `grep` and `wc -l` correctly.

✅ **Checkpoint:** You can create kernel-to-userspace interfaces via `procfs`, `sysfs`, and `debugfs`.

LAB 10: Kernel Timers 🍓

🎯 Mission: "Ticking Inside the Kernel"

Story: Many kernel subsystems need to schedule work for the future — polling a device, implementing timeouts, or periodic housekeeping. Kernel timers are the mechanism.

📖 Background

Kernel timer types:

API	Type	Precision	Use Case
<code>timer_list</code>	Software timer	jiffies (~1-10ms)	General-purpose delayed work
<code>hrtimer</code>	High-resolution timer	nanoseconds	Precise timing, real-time
<code>jiffies</code>	Tick counter	HZ-dependent	Simple delays, timestamps

```
/* timer_list API */
#include <linux/timer.h>

struct timer_list my_timer;
timer_setup(&my_timer, callback_fn, 0);
```

```

mod_timer(&my_timer, jiffies + msecs_to_jiffies(1000)); /* 1 second */
del_timer_sync(&my_timer);

/* hrtimer API */
#include <linux/hrtimer.h>

struct hrtimer my_hr_timer;
hrtimer_init(&my_hr_timer, CLOCK_MONOTONIC, HRTIMER_MODE_REL);
my_hr_timer.function = hr_callback;
hrtimer_start(&my_hr_timer, ktime_set(0, 500000000), HRTIMER_MODE_REL); /* 500ms */
hrtimer_cancel(&my_hr_timer);

```

❧ Exercise 10.1: Standard Kernel Timers

Objective: Create and manage kernel timers.

Step 1: Examine and build:

```

cd code/kernel_programming/lab10_timers/
cat ktimer_demo.c
make

```

Step 2: Load and observe:

```

sudo insmod ktimer_demo.ko interval_ms=500
dmesg -w # Watch timer ticks in real-time

# After a few seconds, check the tick count
cat /sys/module/ktimer_demo/parameters/interval_ms

# Change interval at runtime
sudo bash -c "echo 200 > /sys/module/ktimer_demo/parameters/interval_ms"
dmesg | tail -10

# Stop
sudo rmmod ktimer_demo
dmesg | tail -5

```

🍷 **Raspberry Pi:** Kernel timers work identically on ARM. The `jiffies` value may have different `HZ` (typically 100 on RPi vs 250/1000 on x86). Check with: `grep CONFIG_HZ /boot/config-$(uname -r)`

❧ Exercise 10.2: High-Resolution Timers

Objective: Use hrtimers for nanosecond-precision timing.

```

cat hrtimer_demo.c
make
sudo insmod hrtimer_demo.ko
dmesg | tail -20

# Observe precise timing vs standard timer
sudo rmmod hrtimer_demo

```

🎮 **Challenge:** Create a module with both a standard timer (1s periodic) and an hrtimer (100ms periodic). Log the actual elapsed time for each tick and compare the jitter.

LAB 11: Kernel Threads

Mission: "Background Workers"

Story: Sometimes a kernel module needs to perform long-running or blocking operations. Kernel threads are the solution — they run in kernel space with full kernel privileges.

Background

```
#include <linux/kthread.h>

/* Create and start a kernel thread */
struct task_struct *thread;
thread = kthread_run(thread_fn, data, "my_thread");

/* Thread function signature */
int thread_fn(void *data) {
    while (!kthread_should_stop()) {
        /* Do work */
        msleep(1000); /* Sleep 1 second */
    }
    return 0;
}

/* Stop the thread */
kthread_stop(thread);
```

❏ Exercise 11.1: Basic Kernel Thread

Step 1: Examine and build:

```
cd code/kernel_programming/lab11_threads/
cat kthread_demo.c
make
```

Step 2: Load and observe:

```
sudo insmod kthread_demo.ko
dmesg -w # Watch thread output

# See the kernel thread in process list
ps aux | grep my_kthread

# Check /proc for the thread
ls /proc/$(pgrep my_kthread) 2>/dev/null

sudo rmmod kthread_demo
dmesg | tail -5
```

 **Raspberry Pi:** Kernel threads work the same on ARM. Use `top` to see your thread's CPU usage.

🔗 Exercise 11.2: Multiple Threads with Synchronization

Step 1: Examine the multi-thread demo:

```
cat kthread_multi.c
make
sudo insmod kthread_multi.ko num_threads=4
dmesg | tail -20

# Watch threads working
ps aux | grep worker_thread

sudo rmmod kthread_multi
dmesg | tail -10
```

🎮 **Challenge:** Create a producer-consumer system using kernel threads and a shared buffer protected by a spinlock. One thread produces data, another consumes it.

LAB 12: Bottom Halves & Softirq 🍓

🎯 Mission: "Deferred Work in the Kernel"

Story: Hardware interrupts must be fast — but what about work that can be deferred? Bottom halves (tasklets, workqueues, softirqs) allow the kernel to defer non-urgent work to a safer execution context.

📖 Background

Interrupt Handling

- └─ Top Half (hardirq) – runs immediately, IRQs disabled
 - └─ Do minimum: acknowledge HW, read urgent data, schedule bottom half
- └─ Bottom Half (deferred) – runs later, IRQs enabled
 - └─ Softirq – static, per-CPU, highest priority (network, block)
 - └─ Tasklet – dynamic, built on softirq, simpler API
 - └─ Workqueue – runs in process context (can sleep!)
 - └─ system_wq – shared default workqueue
 - └─ Custom workqueue – for your driver
 - └─ Delayed work – schedule after a delay
- └─ Threaded IRQ – IRQ handler runs in a kernel thread

Mechanism	Context	Can Sleep?	Concurrency	Use Case
Softirq	Softirq	No	Per-CPU parallel	Networking, block I/O
Tasklet	Softirq	No	Serialized (single CPU)	Simple deferred work
Workqueue	Process	Yes	Configurable	Complex work, I/O, allocation
Threaded IRQ	Thread	Yes	Per-IRQ thread	Modern driver pattern

🔗 Exercise 12.1: Tasklets

Step 1: Examine and build:

```
cd code/kernel_programming/lab12_bottom_halves/
cat tasklet_demo.c
make
```

Step 2: Load and test:

```
sudo insmod tasklet_demo.ko
dmesg | tail -10

# Trigger the tasklet (via a proc write or timer)
echo "trigger" | sudo tee /proc/tasklet_trigger
dmesg | tail -5

sudo rmmod tasklet_demo
```

🔗 Exercise 12.2: Workqueues

Step 1: Examine and test the workqueue demo:

```
cat workqueue_demo.c
make
sudo insmod workqueue_demo.ko
dmesg | tail -15

# Observe: workqueue handler runs in process context (can sleep!)
# Check which kernel thread runs the work
dmesg | grep "workqueue" | tail -10

sudo rmmod workqueue_demo
```

🍓 **Raspberry Pi:** Bottom halves are essential on RPi for GPIO interrupt handling. The concepts here directly apply to GPIO drivers.

🔗 Exercise 12.3: Softirq (Observation)

Step 1: Observe softirqs on your system:

```
# Softirq statistics
cat /proc/softirqs

# Watch softirq activity
watch -n 1 cat /proc/softirqs

# Softirq threads
ps aux | grep ksoftirqd
```

🎮 **Challenge:** Create a module that uses all three: a timer fires → triggers a tasklet → tasklet schedules a work item → work item logs the timing through the chain.

LAB 13: Memory Allocation in the Kernel 🍓

🎯 **Mission: "Managing Precious Kernel Memory"**

Story: Memory management in the kernel is critical and completely different from user space. There's no virtual memory safety net — allocate wrong and the whole system crashes.

Background

Kernel Memory Allocators:

Allocator	Function	Use Case	Notes
kmalloc	<code>kmalloc(size, flags)</code>	Small allocations (<128 KB)	Physically contiguous, fast
kzalloc	<code>kzalloc(size, flags)</code>	Same + zeroed	Preferred over <code>kmalloc+memset</code>
vmalloc	<code>vmalloc(size)</code>	Large allocations	Virtually contiguous only
kvmalloc	<code>kvmalloc(size, flags)</code>	Flexible size	Tries <code>kmalloc</code> , falls back to <code>vmalloc</code>
Page alloc	<code>alloc_pages(flags, order)</code>	Page-level allocation	2^{order} pages
Slab/SLUB	<code>kmem_cache_create()</code>	Repeated same-size allocs	Object caching, very fast

GFP Flags (Get Free Pages):

Flag	Context	Can Sleep?	Use
<code>GFP_KERNEL</code>	Process context	Yes	Normal allocation
<code>GFP_ATOMIC</code>	Interrupt/spinlock	No	Cannot sleep
<code>GFP_DMA</code>	DMA-capable memory	—	Device I/O
<code>GFP_HIGHUSER</code>	High memory, user	Yes	User pages

❧ Exercise 13.1: kmalloc and Friends

Step 1: Examine and build:

```
cd code/kernel_programming/lab13_memory/
cat kmalloc_demo.c
make
```

Step 2: Load and observe:

```
sudo insmod kmalloc_demo.ko
dmesg | tail -30

# Check kernel memory stats
cat /proc/meminfo | head -10
cat /proc/slabinfo | head -20

sudo rmmod kmalloc_demo
```

🍷 **Raspberry Pi:** RPi has limited RAM (1-8 GB). Memory allocation exercises are especially relevant — observe how `kmaLLoc` vs `vmmaLLoc` behave under memory pressure.

❧ Exercise 13.2: Slab Allocator — Custom Caches

Step 1: Examine and test:

```
cat slab_demo.c
make
sudo insmod slab_demo.ko

# Observe your custom slab cache
sudo cat /proc/slabinfo | grep my_cache

# See slab statistics
sudo slabtop -o | head -30

sudo rmmod slab_demo
```

🎮 **Challenge:** Create a module that benchmarks: `kmalloc` vs `vmalloc` vs `kmem_cache_alloc` for 10,000 allocations of the same size. Log allocation/deallocation times.

LAB 14: Synchronization Primitives 🍓

🔧 Mission: "Guarding Shared Data"

Story: In a multi-CPU kernel, multiple threads and interrupt handlers access shared data simultaneously. Without proper synchronization, data corruption and crashes are inevitable.

📖 Background

Synchronization mechanisms:

Mechanism	Context	Preemption	IRQs	Use Case
Atomic ops	Any	—	—	Simple counters, flags
Spinlock	Non-sleeping	Disabled	Enabled	Short critical sections
spin_lock_irqsave	Any	Disabled	Disabled	Data shared with IRQ handler
Mutex	Process only	—	—	Long critical sections (can sleep)
Semaphore	Process only	—	—	Counting resource access
RW Lock	Mixed	Disabled	—	Read-heavy data
RCU	Special	—	—	Read-mostly data structures
Completion	Process	—	—	Thread synchronization

🔗 Exercise 14.1: Spinlocks and Mutexes

Step 1: Examine and build:

```
cd code/kernel_programming/lab14_sync/
cat sync_demo.c
make
```

Step 2: Load and observe:

```
sudo insmod sync_demo.ko
dmesg | tail -30
```

```
# Module demonstrates:
# - Atomic operations (atomic_t)
# - Spinlock protecting a counter
# - Mutex protecting a buffer
# - Proper lock ordering
```

```
sudo rmmod sync_demo
```

 **Raspberry Pi:** RPi 3B+ and 4 are multi-core (4 CPUs). Synchronization bugs are real on RPi — spinlocks and mutexes are essential. Test with `stress -c 4` running in background.

❧ Exercise 14.2: RCU (Read-Copy-Update)

Step 1: Examine the RCU demo:

```
cat rcu_demo.c
make
sudo insmod rcu_demo.ko
dmesg | tail -20

sudo rmmod rcu_demo
```

❧ Exercise 14.3: Completions

```
cat completion_demo.c
make
sudo insmod completion_demo.ko
dmesg | tail -15

sudo rmmod completion_demo
```

 **Challenge:** Create a module with a race condition (incrementing a shared counter from multiple kthreads without locking). Then fix it with: (a) atomic_t, (b) spinlock, (c) mutex. Compare the final counter values and performance.

LAB 15: Signals & Barriers

Mission: "Communication and Ordering"

Story: Kernel code needs to handle signals from userspace and enforce memory ordering across CPUs. These are subtle but critical for correctness.

Background — Signals in the Kernel

In the kernel, signals are sent to processes, not handled by modules directly. But modules need to be aware of signals (e.g., when a blocked read is interrupted by SIGINT).

```
/* Check if current process has a pending signal */
if (signal_pending(current)) {
```

```
    return -ERESTARTSYS; /* Let userspace retry */
}

/* Send a signal to a process */
send_sig(SIGKILL, task, 0);
send_sig_info(SIGUSR1, &info, task);
```

Background — Memory Barriers

```
/* Compiler barrier – prevent compiler reordering */
barrier();

/* CPU memory barriers */
smp_mb(); /* Full memory barrier (read + write) */
smp_rmb(); /* Read memory barrier */
smp_wmb(); /* Write memory barrier */

/* For MMIO */
readl() / writel() /* Include implicit barriers */
mb(), rmb(), wmb() /* Hardware memory barriers */
```

❧ Exercise 15.1: Signal-Aware Kernel Code

Step 1: Examine and build:

```
cd code/kernel_programming/lab15_signals_barriers/
cat signal_aware.c
make
```

Step 2: Test signal handling:

```
sudo insmod signal_aware.ko

# Read from the device (will block)
cat /dev/signal_test &
CAT_PID=$!

# Send a signal to interrupt it
kill -SIGINT $CAT_PID
dmesg | tail -5

sudo rmmod signal_aware
```

❧ Exercise 15.2: Memory Barriers

Step 1: Examine the barrier demo:

```
cat barrier_demo.c
make
sudo insmod barrier_demo.ko
dmesg | tail -20

sudo rmmod barrier_demo
```

 **Challenge:** Create a lockless ring buffer between a producer kthread and consumer kthread, using only `smp_wmb()` and `smp_rmb()` for synchronization (no locks). This is the pattern used by the kernel's kfifo.

LAB 16: Kernel Data Structures

Mission: "The Kernel's Collections"

Story: The Linux kernel has its own implementation of common data structures, highly optimized for kernel use. Understanding them is essential for writing any non-trivial driver.

Background

Data Structure	Header	API Pattern
Linked List	<linux/list.h>	list_add(), list_for_each_entry()
Hash Table	<linux/hashtable.h>	hash_add(), hash_for_each()
Red-Black Tree	<linux/rbtree.h>	rb_insert_color(), rb_first()
Radix Tree/XArray	<linux/xarray.h>	xa_store(), xa_load()
Bitmap	<linux/bitmap.h>	set_bit(), test_bit()
Kfifo	<linux/kfifo.h>	kfifo_put(), kfifo_get()
IDR/IDA	<linux/idr.h>	idr_alloc(), idr_find()

🔗 Exercise 16.1: Kernel Linked List

The kernel linked list is **intrusive** — the list node is embedded inside your structure:

```
struct my_item {
    int data;
    char name[32];
    struct list_head list; /* Embedded list node */
};

LIST_HEAD(my_list); /* Declare and initialize list head */

/* Add item */
struct my_item *item = kmalloc(sizeof(*item), GFP_KERNEL);
item->data = 42;
list_add(&item->list, &my_list);

/* Iterate */
struct my_item *entry;
list_for_each_entry(entry, &my_list, list) {
    pr_info("Item: %d\n", entry->data);
}
```

Step 1: Examine and build:

```
cd code/kernel_programming/lab16_data_structures/
cat klist_demo.c
```

```
make
```

Step 2: Load and observe:

```
sudo insmod klist_demo.ko
dmesg | tail -30

# Module demonstrates: add, remove, iterate, safe iteration during removal

sudo rmmod klist_demo
```

 **Raspberry Pi:** Kernel data structures are architecture-independent. All demos work on RPi.

❧ Exercise 16.2: Hash Table and kfifo

```
cat hashtable_demo.c
cat kfifo_demo.c
make

sudo insmod hashtable_demo.ko
dmesg | tail -20
sudo rmmod hashtable_demo

sudo insmod kfifo_demo.ko
dmesg | tail -15
sudo rmmod kfifo_demo
```

 **Challenge:** Build a "kernel phone book" module: add entries via sysfs write, look up by name via sysfs read, backed by a hash table. Support at least 100 entries.

LAB 17: Interrupts & Threaded IRQ

Mission: "Handling Hardware Events"

Story: Hardware devices signal the CPU via interrupts. Writing a correct interrupt handler is one of the most critical skills in driver development — get it wrong and the system locks up.

Background

```
#include <linux/interrupt.h>

/* Classic IRQ handler */
irqreturn_t my_irq_handler(int irq, void *dev_id) {
    /* Top half — fast, no sleeping! */
    /* Read device status, acknowledge interrupt */
    return IRQ_HANDLED; /* or IRQ_NONE if not our interrupt */
}

/* Register IRQ */
request_irq(irq_num, my_irq_handler, IRQF_SHARED, "my_device", dev);

/* Threaded IRQ — top half + threaded bottom half */
```



```
request_threaded_irq(irq_num,
    quick_handler,    /* Top half (hardirq) – can be NULL */
    thread_fn,        /* Bottom half (thread context – can sleep!) */
    IRQF_SHARED, "my_device", dev);

/* Free IRQ */
free_irq(irq_num, dev);
```

IRQ Flags:

Flag	Purpose
IRQF_SHARED	IRQ line shared with other devices
IRQF_TRIGGER_RISING	Trigger on rising edge
IRQF_TRIGGER_FALLING	Trigger on falling edge
IRQF_ONESHOT	Keep IRQ disabled until thread completes

🔗 Exercise 17.1: Interrupt Handling

Step 1: Examine the interrupt demo:

```
cd code/kernel_programming/lab17_interrupts/
cat irq_demo.c
```

Step 2: Observe interrupts on your system:

```
# Current interrupt counts
cat /proc/interrupts | head -20

# Watch interrupt activity
watch -n 1 "cat /proc/interrupts | head -20"

# IRQ affinity
cat /proc/irq/*/smp_affinity_list 2>/dev/null | head -10
```

Step 3: Build and test (using a software-triggered interrupt):

```
make
sudo insmod irq_demo.ko

# The module demonstrates:
# - Requesting an IRQ (using a shared timer IRQ for demo)
# - Threaded IRQ handler
# - Interrupt accounting

cat /proc/interrupts | grep my_irq

sudo rmmod irq_demo
```

🍷 **Raspberry Pi:** On RPi, you can request real GPIO interrupts! The GPIO pins can trigger on rising/falling edges. See the GPIO driver lab for hands-on interrupt handling with physical buttons.

```
# On RPi, see GPIO IRQs:
cat /proc/interrupts | grep gpio
```

❏ Exercise 17.2: Threaded IRQ

Step 1: Examine the threaded IRQ demo:

```
cat threaded_irq_demo.c
make
sudo insmod threaded_irq_demo.ko

dmesg | tail -10
# Observe: top half runs in IRQ context, thread runs in process context

sudo rmmod threaded_irq_demo
```

🎮 **Challenge:** Create a module that measures interrupt latency: record `ktime_get()` in the top half and again in the threaded handler. Report the distribution of latencies over 100 interrupts.

LAB 18: Character Driver — Old and New Method 🍓

🎯 Mission: "Your First Real Driver"

Story: Character drivers are the most common type of Linux driver. You'll implement one using both the legacy method (manual major/minor allocation) and the modern method (automatic allocation with `cdev`).

📖 Background

Old method (`register_chrdev`):

```
/* Static major number – fragile, can conflict */
#define MY_MAJOR 240

register_chrdev(MY_MAJOR, "mydev", &fops);
/* Creates major 240, all 256 minors */

/* Cleanup */
unregister_chrdev(MY_MAJOR, "mydev");
```

New method (`alloc_chrdev_region` + `cdev`):

```
/* Dynamic major number – preferred */
dev_t dev;
alloc_chrdev_region(&dev, 0, 1, "mydev");
/* Kernel assigns an available major number */

/* cdev – represents one character device */
struct cdev my_cdev;
cdev_init(&my_cdev, &fops);
cdev_add(&my_cdev, dev, 1);

/* Create device node automatically */
```

```

struct class *cl = class_create("my_class");
device_create(cl, NULL, dev, NULL, "mydev0");

/* Cleanup (reverse order) */
device_destroy(cl, dev);
class_destroy(cl);
cdev_del(&my_cdev);
unregister_chrdev_region(dev, 1);

```

File Operations:

```

static struct file_operations fops = {
    .owner    = THIS_MODULE,
    .open     = my_open,
    .release  = my_release,
    .read     = my_read,
    .write    = my_write,
    .unlocked_ioctl = my_ioctl,
    .llseek   = my_llseek,
};

```

❏ Exercise 18.1: Old Method — register_chrdev

Step 1: Examine the old-style driver:

```

cd code/kernel_programming/lab18_chardev/
cat chardev_old.c

```

Step 2: Build, load, create device node manually:

```

make

# Load
sudo insmod chardev_old.ko
dmesg | tail -5

# Create device node MANUALLY (old method requires this)
sudo mknod /dev/oldchar c 240 0
sudo chmod 666 /dev/oldchar

# Test
echo "Hello Driver" > /dev/oldchar
cat /dev/oldchar
dmesg | tail -10

# Cleanup
sudo rm /dev/oldchar
sudo rmmod chardev_old

```

❏ Exercise 18.2: New Method — alloc_chrdev_region + cdev

Step 1: Examine the modern driver:

```
cat chardev_new.c
```

Step 2: Build and test — device node is created automatically:

```
make

sudo insmod chardev_new.ko
dmesg | tail -5

# Device node is auto-created by udev!
ls -la /dev/mychardev*

# Test read/write
echo "Hello Modern Driver" > /dev/mychardev0
cat /dev/mychardev0

# Test with dd
dd if=/dev/mychardev0 bs=32 count=1 2>/dev/null | xxd

# Test multiple opens
cat /dev/mychardev0 &
echo "concurrent" > /dev/mychardev0

# ioctl testing (if implemented)
# Use a custom userspace program for ioctl

sudo rmmod chardev_new
dmesg | tail -10
```

 **Raspberry Pi:** Character drivers work identically on RPi. The auto-created `/dev/` node via udev works out of the box on Raspberry Pi OS.

❧ Exercise 18.3: Character Driver with ioctl

Step 1: Examine the ioctl-enabled driver:

```
cat chardev_ioctl.c
cat test_ioctl.c    # Userspace test program
```

Step 2: Build and test:

```
make

sudo insmod chardev_ioctl.ko

# Build and run the test program
gcc -Wall -o test_ioctl test_ioctl.c
./test_ioctl

dmesg | tail -20
sudo rmmod chardev_ioctl
```

 **Challenge:** Create a character driver that implements a "kernel notepad" — supports read, write, seek, and ioctl for: getting buffer size, clearing the buffer, and getting write count. Include proper locking for concurrent access.

✔ **Checkpoint:** You can create fully functional character drivers with automatic device node creation.



Mini-Project 3: Kernel Module Test Framework

Mission: "The Kernel Unit Tester"

Build a reusable test framework for kernel modules. It should load modules, exercise their interfaces, verify behavior, and produce test reports.

Requirements

1. Test Loader Module (`ktest_framework.ko`)

- Loads as a module
- Exposes `/proc/ktest/run` — write a test name to run it
- Exposes `/proc/ktest/results` — read test results
- Supports registering test cases at compile time

2. Test Macros (`ktest.h`)

```
KTEST_ASSERT_EQ(a, b)
KTEST_ASSERT_NE(a, b)
KTEST_ASSERT_TRUE(cond)
KTEST_ASSERT_NOT_NULL(ptr)
KTEST_EXPECT_EQ(a, b) /* Non-fatal */
```

3. Sample Test Modules

- Test `kmalloc/kfree` cycles
- Test list operations
- Test spinlock correctness
- Test kthread creation

4. Result Format

```
[PASS] test_kmalloc_basic
[PASS] test_list_add_remove
[FAIL] test_spinlock_contention: expected 10000, got 9998
[PASS] test_kthread_lifecycle
=== 3 passed, 1 failed ===
```

Deliverables

```
code/kernel_programming/mini_project3_ktest/
├─ ktest.h           # Test macros and API
├─ ktest_framework.c # Framework module
├─ test_memory.c     # Memory tests
├─ test_list.c       # List tests
├─ test_sync.c       # Synchronization tests
├─ Makefile
└─ README.md
```



Mini-Project 4: Debugfs System Monitor

Mission: "The Kernel Health Dashboard"

Create a kernel module that exposes comprehensive system monitoring data via debugfs, providing real-time insights into kernel internals.

Requirements

1. **CPU Monitor** (`/sys/kernel/debug/sysmon/cpu`)
 - Per-CPU load, context switches, interrupts
 - Scheduling statistics
 - Updated every second via kernel timer
2. **Memory Monitor** (`/sys/kernel/debug/sysmon/memory`)
 - SLAB usage for top 10 caches
 - Page allocation statistics
 - vmalloc usage
3. **Module Monitor** (`/sys/kernel/debug/sysmon/modules`)
 - List of loaded modules with sizes
 - Symbol export counts
 - Dependency information
4. **Event Log** (`/sys/kernel/debug/sysmon/events`)
 - Ring buffer of kernel events
 - Timestamped entries
 - Filterable by severity

Deliverables

```
code/kernel_programming/mini_project4_sysmon/
├─ sysmon_main.c      # Module entry, debugfs setup
├─ sysmon_cpu.c       # CPU monitoring
├─ sysmon_memory.c    # Memory monitoring
├─ sysmon_events.c    # Event ring buffer
├─ sysmon.h           # Shared definitions
├─ Makefile
└─ README.md
```



Capstone Project: Multi-Interface Character Device with Monitoring

Mission: "The Complete Kernel Driver"

Build a production-quality character driver that demonstrates all Part 1 & Part 2 concepts in a single, cohesive project.

Phase 1: Core Character Driver

- Dynamic major/minor allocation (new method)
- Supports open, read, write, ioctl, lseek
- Internal ring buffer (using kfifo)
- Multiple device nodes (e.g., `/dev/mypipe0` , `/dev/mypipe1`)

Phase 2: Kernel Interfaces

- **procfs:** `/proc/mypipe/stats` — read/write counters, buffer usage
- **sysfs:** `/sys/class/mypipe/mypipe0/buffer_size` — configurable
- **debugfs:** `/sys/kernel/debug/mypipe/` — internal state dump, ring buffer hexdump

Phase 3: Concurrency & Timing

- Proper locking (mutex for process context, spinlock for stats)
- Wait queue for blocking reads on empty buffer
- Kernel timer for periodic stats logging
- Kernel thread for buffer housekeeping

Phase 4: Interrupt Simulation

- Simulate interrupts via hrtimer
- Threaded IRQ handler pattern
- Tasklet for deferred data processing
- Workqueue for periodic maintenance

Phase 5: Data Structures & Memory

- kfifo for the ring buffer
- Hash table for per-client tracking
- Custom slab cache for message objects
- Memory-barrier-protected status flags

Phase 6: Testing

- Userspace test program with ioctl
- Multi-threaded reader/writer stress test
- Module parameter for buffer size
- Error injection for robustness testing

Deliverables

```
code/kernel_programming/capstone_chardev/
├─ mypipe_core.c      # Main driver (open, read, write, ioctl)
├─ mypipe_procfs.c    # /proc interface
├─ mypipe_sysfs.c     # /sys interface
├─ mypipe_debugfs.c   # debugfs interface
├─ mypipe_timer.c     # Timer and deferred work
├─ mypipe.h           # Shared structures and definitions
├─ test_mypipe.c      # Userspace test program
├─ stress_test.c      # Multi-threaded stress test
├─ Makefile
└─ README.md
```

Evaluation Criteria

- All device operations work correctly
- No kernel oops under stress testing
- Proper cleanup on module unload (no resource leaks)
- All three interfaces (procfs/sysfs/debugfs) functional
- Blocking read works with signal handling (`-ERESTARTSYS`)
- Works on both x86_64 VM and Raspberry Pi (🍷)

PART 3 — Advanced Drivers, Subsystems & Device Tree

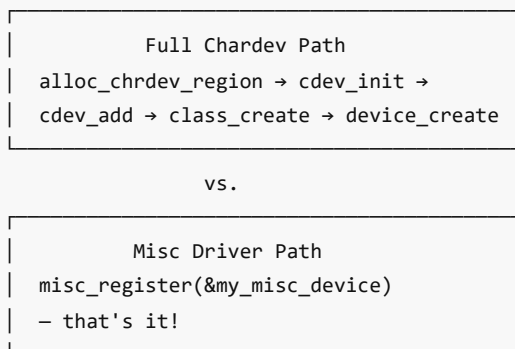
LAB 19: Misc Driver 🍓

🎯 Mission: "The Simplest Driver"

Story: Your team needs a quick driver for a small custom device. The full character device framework feels heavy. A senior engineer mentions the **misc driver** framework — a lightweight wrapper over the character device API that auto-allocates minor numbers and creates device nodes. Your job: build one and understand why it exists.

📖 Background

The **misc (miscellaneous) device** framework simplifies character driver creation:



- All misc devices share **major 10** with auto-assigned minor numbers
- Device node `/dev/<name>` is automatically created
- Uses `struct miscdevice` with `struct file_operations`
- Ideal for simple, single-instance devices

✅ Challenges

Challenge 19.1 — Basic Misc Driver

Write a misc driver that creates `/dev/mymisc` :

1. Define a `struct miscdevice` with `.minor = MISC_DYNAMIC_MINOR` , `.name = "mymisc"` , `.fops` pointing to your file operations
2. Implement `open` , `release` , `read` , `write` operations
3. Use a kernel buffer (1 KB) to store data written by userspace
4. Read should return whatever was last written

```
# After loading the module:
echo "Hello Misc" > /dev/mymisc
cat /dev/mymisc      # Should print "Hello Misc"
```

Challenge 19.2 — Misc Driver with Multiple Instances

Extend your misc driver to support private data per open file descriptor:

1. Allocate per-fd data in `open()` using `file->private_data`
2. Each open gets its own independent buffer
3. Test with two simultaneous reads/writes from different terminals

Challenge 19.3 — Compare Misc vs. Full Chardev

1. Count the lines of boilerplate code for `misc_register()` vs. full cdev path
2. Examine `/proc/misc` to see all registered misc devices
3. Run `ls -la /dev/mymisc` and note the major number is **10**

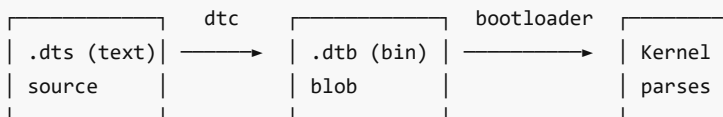
LAB 20: Device Tree 🍓

🔧 Mission: "The Hardware Blueprint"

Story: You've been given a custom board with unfamiliar peripherals. On x86, the BIOS/ACPI tells the kernel what hardware exists. On ARM (Raspberry Pi), there's no BIOS — instead, a **Device Tree** (DT) describes the hardware in a structured data file. You must learn to read, modify, and create device tree nodes.

📖 Background

Device Tree Flow:



- **DTS** (Device Tree Source) — human-readable text format
- **DTB** (Device Tree Blob) — compiled binary, loaded by bootloader
- **DTC** (Device Tree Compiler) — compiles `.dts` → `.dtb`
- **DTSI** — include files (shared across boards)
- **Device Tree Overlay** — runtime modifications to the base DT (used heavily on RPi)

Key concepts:

- **Nodes** represent devices or buses (e.g., `i2c@7e804000`)
- **Properties** describe configuration (`reg`, `compatible`, `interrupts`, `status`)
- `compatible` string links the DT node to a driver
- `phandle` references between nodes (e.g., a device referencing its clock)

✅ Challenges

Challenge 20.1 — Explore the Live Device Tree

```
# View the live device tree (from /proc or /sys)
ls /proc/device-tree/          # or /sys/firmware/devicetree/base/
# Read a property
cat /proc/device-tree/model
cat /proc/device-tree/compatible

# Dump entire tree in human-readable form
dtc -I fs /proc/device-tree/ 2>/dev/null | head -100

# On RPi:
cat /proc/device-tree/model    # "Raspberry Pi 4 Model B Rev 1.4"
```

1. Find the `compatible` string for your root node
2. List all nodes under `/proc/device-tree/soc/`
3. Find an I2C controller node and read its `reg` property
4. Find the `status` property — is it "okay" or "disabled" ?

Challenge 20.2 — Decompile and Examine a DTB 🍓

```
# On RPi, the DTB is in /boot/  
ls /boot/*.dtb      # or /boot/firmware/*.dtb  
dtc -I dtb -O dts /boot/bcm2711-rpi-4-b.dtb > rpi4.dts  
less rpi4.dts  
  
# On x86 VM (may use ACPI instead):  
ls /boot/dtbs/ 2>/dev/null || echo "x86 typically uses ACPI, not DT"
```

1. Decompile the DTB to DTS format
2. Find the UART node — note `compatible`, `reg`, `interrupts` properties
3. Find a GPIO controller node
4. Identify how the `pinctrl` subsystem references pin configurations

Challenge 20.3 — Write a Device Tree Overlay 🍓

Create an overlay that defines a virtual device:

```
// my_overlay.dts  
/dts-v1/;  
/plugin/;  
  
/ {  
    compatible = "brcm,bcm2711"; /* RPi 4 */  
  
    fragment@0 {  
        target-path = "/";  
        __overlay__ {  
            my_virtual_device {  
                compatible = "lab,mydevice";  
                status = "okay";  
                my-custom-prop = <42>;  
                label = "Lab20 Virtual Device";  
            };  
        };  
    };  
};
```

1. Compile: `dtc -@ -I dts -O dtb -o my_overlay.dtbo my_overlay.dts`
2. On RPi: copy to `/boot/overlays/` and add `dtoverlay=my_overlay` to `config.txt`
3. Reboot and verify: `cat /proc/device-tree/my_virtual_device/compatible`
4. On non-RPi: use `dtoverlay` command or ConfigFS for runtime overlays

Challenge 20.4 — Device Tree Bindings

1. Browse `Documentation/devicetree/bindings/` in kernel source
2. Read the binding document for a familiar driver (e.g., `gpio-keys`)
3. Write a DT node that matches the `gpio-keys` binding schema
4. Understand `#address-cells`, `#size-cells`, and `ranges`

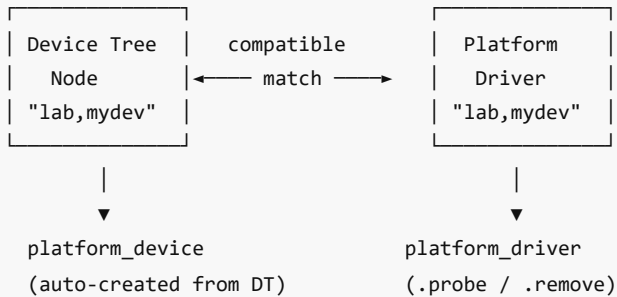
LAB 21: Platform Driver 🍓

🎯 Mission: "Matching Driver to Device"

Story: In the kernel, hardware devices and their drivers are decoupled. The **platform bus** is a virtual bus that matches devices (described in DT or board files) with drivers using the `compatible` string. You must write a platform driver that binds to your device tree node.

Background

Platform Driver Model:



- **platform_device** — created from DT or board code
- **platform_driver** — kernel module that binds to it
- **Probe** — called when a match is found
- **Remove** — called on unbind or module unload
- **of_match_table** — array of `compatible` strings the driver supports

Challenges

Challenge 21.1 — Minimal Platform Driver

Write a platform driver that matches `"lab,mydevice"` :

```
static const struct of_device_id my_of_match[] = {
    { .compatible = "lab,mydevice" },
    { /* sentinel */ }
};
MODULE_DEVICE_TABLE(of, my_of_match);

static struct platform_driver my_platform_driver = {
    .probe = my_probe,
    .remove = my_remove,
    .driver = {
        .name = "my_platform_driver",
        .of_match_table = my_of_match,
    },
};
module_platform_driver(my_platform_driver);
```

1. Implement `probe()` — log device binding, read DT properties using `of_property_read_*`
2. Implement `remove()` — log unbinding
3. Load the module and check `dmesg` for probe message
4. Without DT: use `platform_device_register()` to create a device manually for testing

Challenge 21.2 — Read Device Tree Properties in Probe

In your probe function:

1. Read an integer property: `of_property_read_u32(np, "my-custom-prop", &val)`
2. Read a string property: `of_property_read_string(np, "label", &str)`
3. Check if a boolean property exists: `of_property_read_bool(np, "my-flag")`
4. Get the device node name: `dev_name(&pdev->dev)`

5. Use `dev_info()` instead of `pr_info()` (associates logs with the device)

Challenge 21.3 — Platform Driver with Resources

1. Add `reg` and `interrupts` properties to your DT node
2. In probe, get memory resource: `platform_get_resource(pdev, IORESOURCE_MEM, 0)`
3. Map it: `devm_ioremap_resource(&pdev->dev, res)`
4. Get IRQ: `platform_get_irq(pdev, 0)`
5. Use `devm_*` (managed) APIs for automatic cleanup

Challenge 21.4 — Manual Platform Device (No DT)

For systems without DT (x86 VMs), create a platform device programmatically:

```
/* In a separate "board" module */
static struct platform_device my_pdev = {
    .name = "my_platform_driver",
    .id   = -1,
};
platform_device_register(&my_pdev);
```

1. Create two modules: one registers the device, other registers the driver
2. Load the device module first, then the driver — observe probe
3. Unload in reverse order — observe remove

LAB 22: Kernel Module Context & Idle Thread

Mission: "Understanding Execution Context"

Story: When your kernel code runs, it runs in one of several contexts — process context, interrupt context, softirq context, or even the idle thread. Knowing where your code executes determines what operations are safe. You must map these contexts and understand the idle thread.

Background

Kernel Execution Contexts:

Process Context (can sleep)
- System calls, ioctl, read/write
- Workqueues, kernel threads
- current != NULL, has mm_struct
Interrupt Context (CANNOT sleep)
- Hardirq handlers
- Softirq, tasklets
- in_interrupt() returns true
Idle Thread (CPU idle, PID 0)
- swapper/N (one per CPU)
- Runs when no other task is runnable
- Calls cpuidle governor
- Can be preempted by any interrupt

✓ Challenges

Challenge 22.1 — Context Detection Module

Write a module that detects and logs its execution context:

```
void report_context(const char *label) {
    pr_info("[context] %s: PID=%d, comm=%s, CPU=%d, "
           "in_interrupt=%d, in_softirq=%d, in_hardirq=%d, "
           "preemptible=%d, can_sleep=%d\n",
           label, current->pid, current->comm,
           smp_processor_id(),
           !!in_interrupt(), !!in_softirq(), !!in_hardirq(),
           preemptible(), !in_atomic());
}
```

1. Call `report_context()` from module init (process context)
2. Call it from a timer callback (softirq context)
3. Call it from a workqueue handler (process context)
4. Compare the outputs — note what changes

Challenge 22.2 — Examine Idle Threads

```
# View idle threads (one per CPU, PID 0)
ps -eo pid,comm,psr | grep swapper
# or
cat /proc/0/status 2>/dev/null || echo "PID 0 not visible in /proc"

# Check CPU idle statistics
cat /proc/stat | head -5
cat /sys/devices/system/cpu/cpu0/cpuidle/state0/name
cat /sys/devices/system/cpu/cpu0/cpuidle/state0/time
```

1. Count how many idle threads exist (one per CPU)
2. Read idle time from `/proc/stat`
3. Examine cpuidle states for your CPU
4. Write a module that logs when it detects it's running as the idle task (check `current->pid == 0` — though modules won't normally run there)

Challenge 22.3 — Module Init vs. Runtime Context

1. In `module_init`, check `current->comm` — it should be `insmod` or `modprobe`
2. Create a `/proc` entry; when read, check `current->comm` — it's the reading process
3. Verify that `current->mm != NULL` in process context (user process has address space)
4. Observe that `current->mm == NULL` for kernel threads

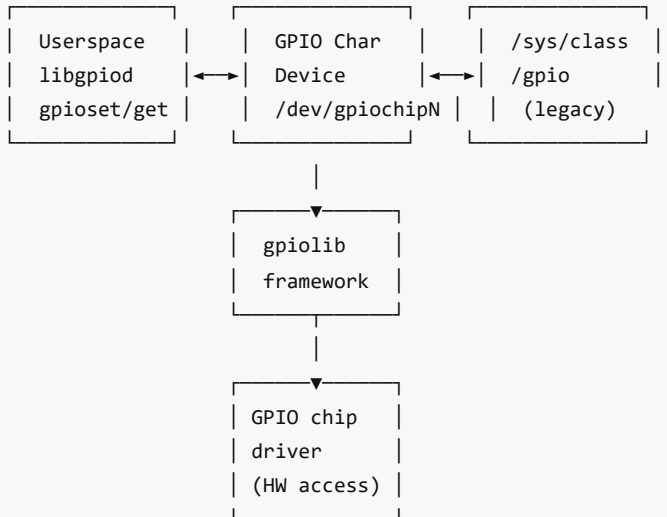
LAB 23: GPIO Driver

Mission: "Controlling the Pins"

Story: Your Raspberry Pi has 40 GPIO pins waiting to be controlled. You must write a kernel driver that exports GPIO pins, handles input/output, and responds to edge-triggered interrupts. This is the foundation for all embedded Linux work.

Background

GPIO Subsystem Architecture:



- **Legacy sysfs GPIO** (/sys/class/gpio/) — deprecated but still common
- **New chardev GPIO** (/dev/gpiochipN) — preferred, uses libgpiod
- **In-kernel GPIO API** — gpio_request , gpio_direction_input/output , gpio_get_value , gpio_set_value
- **Descriptor-based API** (preferred) — gpiod_get , gpiod_direction_input , gpiod_set_value

✅ Challenges

Challenge 23.1 — GPIO from Userspace 🍓

```
# Install libgpiod tools
sudo apt install -y gpiod

# List GPIO chips
gpiodetect

# List pins on chip 0
gpioinfo gpiochip0

# Toggle GPIO 17 (RPI physical pin 11) — connect an LED!
gpioset gpiochip0 17=1    # ON
gpioset gpiochip0 17=0    # OFF

# Read GPIO 27 (connect a button with pull-up)
gpioget gpiochip0 27

# Monitor edges (button press)
gpiomon --falling-edge gpiochip0 27
```

1. Connect an LED to GPIO 17 (with 330Ω resistor)
2. Toggle it using gpioset
3. Connect a button to GPIO 27 with pull-up
4. Monitor button presses with gpiomon

Challenge 23.2 — GPIO Kernel Driver (Descriptor API) 🍓

Write a platform driver that controls GPIO from kernel space:

```
#include <linux/gpio/consumer.h>
```

```

struct gpio_desc *led_gpio;

/* In probe: */
led_gpio = devm_gpiod_get(&pdev->dev, "led", GPIOD_OUT_LOW);
if (IS_ERR(led_gpio))
    return PTR_ERR(led_gpio);

gpiod_set_value(led_gpio, 1); /* LED ON */

```

Device tree entry:

```

my_led_device {
    compatible = "lab,gpio-demo";
    led-gpios = <&gpio 17 GPIO_ACTIVE_HIGH>;
    status = "okay";
};

```

1. Write the platform driver with probe/remove
2. Create a DT overlay for RPi (or use manual platform_device on x86)
3. Add a sysfs attribute to control the LED from userspace
4. Toggle the LED periodically using a kernel timer

Challenge 23.3 — GPIO Interrupt (Edge Detection) 🍓

```

int irq = gpiod_to_irq(button_gpio);
devm_request_threaded_irq(&pdev->dev, irq, NULL, button_isr,
    IRQF_TRIGGER_FALLING | IRQF_ONESHOT, "button_irq", priv);

```

1. Request a GPIO for input (button on GPIO 27)
2. Get its IRQ number with `gpiod_to_irq()`
3. Register a threaded IRQ handler for falling edge
4. In the ISR, toggle the LED and log the event
5. Add debouncing (ignore events within 200ms of last event)

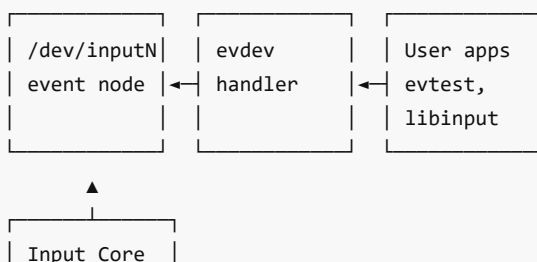
LAB 24: Input Driver 🍓

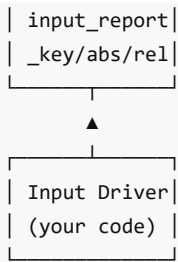
🎯 Mission: "Speaking the Input Language"

Story: Your GPIO button works, but applications like `evtest` can't see it. To make your button visible to the standard Linux input stack, you must register it as an **input device**. This lab teaches the input subsystem — the framework behind keyboards, mice, touchscreens, and game controllers.

📖 Background

Input Subsystem:





- **Input device** — `struct input_dev` , registered with `input_register_device()`
- **Events** — `input_report_key()` , `input_report_abs()` , `input_report_rel()`
- **Event types** — `EV_KEY` (buttons), `EV_ABS` (touchscreen), `EV_REL` (mouse)
- `input_sync()` — marks end of a batch of events

✅ Challenges

Challenge 24.1 — Virtual Keyboard Driver

Create an input driver that generates key events from a timer:

1. Allocate: `input_allocate_device()`
2. Set capabilities: `set_bit(EV_KEY, dev->evbit)` , `set_bit(KEY_A, dev->keybit)`
3. Register: `input_register_device(dev)`
4. From a timer, send: `input_report_key(dev, KEY_A, 1); input_sync(dev);`
5. Test with `evtest /dev/input/eventN`

Challenge 24.2 — GPIO Button as Input Device 🍓

Combine Lab 23 GPIO interrupt with the input subsystem:

1. In probe, create an input device for `EV_KEY / KEY_ENTER`
2. In the button ISR, report `input_report_key()` + `input_sync()`
3. Use `evtest` to verify button presses appear as keyboard events
4. Compare with the built-in `gpio-keys` driver (`drivers/input/keyboard/gpio_keys.c`)

Challenge 24.3 — Simple Joystick/Accelerometer Input

Create an input driver that reports `EV_ABS` events:

1. Set up `input_set_abs_params(dev, ABS_X, -100, 100, 0, 0)`
2. Same for `ABS_Y`
3. From a timer, generate sine-wave X/Y values
4. Test with `evtest` — you should see absolute axis events

LAB 25: I2C Driver 🍓

🎯 Mission: "Talking on Two Wires"

Story: An I2C sensor is connected to your board. I2C is a 2-wire bus (SDA + SCL) used by thousands of sensors, EEPROMs, and displays. You must write a kernel I2C client driver that communicates with the device, reads data, and exposes it to userspace.

📖 Background

I2C Stack:



I2C Client Driver	Your driver (e.g., sensor, EEPROM) probe() called when DT match found
I2C Core	i2c_transfer(), i2c_smbus_read_*()
I2C Adapter Driver	Bus controller driver (bcm2835-i2c on RPi) Handles actual SDA/SCL signaling
Hardware	I2C bus, pull-ups, devices

- **I2C Adapter** — bus controller (one per I2C bus)
- **I2C Client** — device on the bus (identified by address, e.g., 0x68)
- **SMBus** — subset of I2C, simpler read/write functions
- `i2c_smbus_read_byte_data(client, reg)` — read one byte from register
- `i2c_smbus_write_byte_data(client, reg, val)` — write one byte

✓ Challenges

Challenge 25.1 — Detect I2C Devices 🍓

```
# Enable I2C on RPi (if not done):
sudo raspi-config  # Interface Options → I2C → Enable

# Install tools
sudo apt install -y i2c-tools

# Scan I2C bus 1 (RPi default)
sudo i2cdetect -y 1

# Read a register from a device (e.g., address 0x68)
sudo i2cget -y 1 0x68 0x75  # WHO_AM_I register on MPU6050
```

1. Connect an I2C device (e.g., MPU6050, BMP280, OLED SSD1306, or AT24 EEPROM)
2. Scan the bus and find the device address
3. Read the device's WHO_AM_I / ID register
4. Read/write a few registers manually with `i2cget` / `i2cset`

Challenge 25.2 — I2C Client Driver 🍓

Write a kernel I2C driver for your sensor:

```
static const struct i2c_device_id my_i2c_id[] = {
    { "my_i2c_sensor", 0 },
    { }
};
MODULE_DEVICE_TABLE(i2c, my_i2c_id);

static const struct of_device_id my_i2c_of_match[] = {
    { .compatible = "lab,my-i2c-sensor" },
    { }
};
MODULE_DEVICE_TABLE(of, my_i2c_of_match);

static struct i2c_driver my_i2c_driver = {
    .driver = {
        .name = "my_i2c_sensor",
```

```

        .of_match_table = my_i2c_of_match,
    },
    .probe = my_i2c_probe,
    .remove = my_i2c_remove,
    .id_table = my_i2c_id,
};
module_i2c_driver(my_i2c_driver);

```

1. Implement probe: read device ID register, allocate private data
2. Create a sysfs attribute that reads a sensor register
3. DT entry:

```

&i2c1 {
    my_sensor@68 {
        compatible = "lab,my-i2c-sensor";
        reg = <0x68>;
        status = "okay";
    };
};

```

4. On x86/VM without I2C: use `i2c-stub` module for testing

Challenge 25.3 — I2C with Regmap

Rewrite your I2C driver using the `regmap` API (covered in Lab 33):

1. Initialize: `devm_regmap_init_i2c(client, &config)`
2. Read: `regmap_read(regmap, reg, &val)`
3. Write: `regmap_write(regmap, reg, val)`
4. Bulk read: `regmap_bulk_read(regmap, start_reg, buf, count)`
5. Compare code size with raw SMBus calls

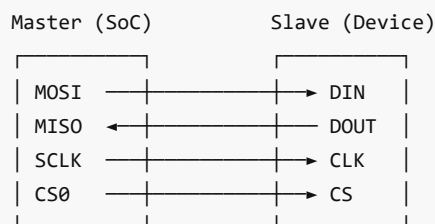
LAB 26: SPI Driver 🍓

🎯 Mission: "High-Speed Serial"

Story: Your project has an SPI flash chip and an SPI display. SPI is faster than I2C (tens of MHz) but uses more wires (MOSI, MISO, SCLK, CS). You must write an SPI driver and understand the SPI framework.

📖 Background

SPI Bus:



SPI Modes (CPOL, CPHA):

- Mode 0: CPOL=0, CPHA=0 (most common)
- Mode 1: CPOL=0, CPHA=1

Mode 2: CPOL=1, CPHA=0

Mode 3: CPOL=1, CPHA=1

✓ Challenges

Challenge 26.1 — SPI Userspace Access 🍓

```
# Enable SPI on RPi:
sudo raspi-config # Interface Options → SPI → Enable

# SPI loopback test (connect MOSI to MISO)
sudo apt install -y spi-tools
# Create a test device
ls /dev/spidev*
# Loopback test
spi-pipe -d /dev/spidev0.0 -s 1000000 < /dev/urandom | hexdump -C | head
```

1. Enable SPI and verify `/dev/spidev0.0` exists
2. Wire MOSI to MISO for loopback
3. Send data and verify it comes back

Challenge 26.2 — SPI Kernel Driver 🍓

Write an SPI client driver:

```
static int my_spi_probe(struct spi_device *spi)
{
    u8 tx_buf[2] = { 0x9F, 0x00 }; /* Read JEDEC ID command */
    u8 rx_buf[4] = { 0 };
    struct spi_transfer xfer = {
        .tx_buf = tx_buf,
        .rx_buf = rx_buf,
        .len     = 4,
    };
    struct spi_message msg;

    spi_message_init(&msg);
    spi_message_add_tail(&xfer, &msg);
    spi_sync(spi, &msg);

    dev_info(&spi->dev, "JEDEC ID: %02x %02x %02x\n",
             rx_buf[1], rx_buf[2], rx_buf[3]);
    return 0;
}
```

1. Write probe/remove functions
2. Use `spi_sync_transfer()` for simplified transfers
3. Set SPI mode and speed: `spi->mode = SPI_MODE_0; spi->max_speed_hz = 1000000;`
4. Add DT node with `spi-max-frequency`, `reg` (CS number)

Challenge 26.3 — SPI with DMA (Large Transfers)

1. Use `spi_async()` for non-blocking transfers with completion callback
 2. Compare performance of `spi_sync()` vs. `spi_async()` for 4KB transfers
 3. Check if your SPI controller supports DMA: `cat /sys/class/spi_master/spi0/dma`
-

LAB 27: I2C from Userspace 🍓

🎯 Mission: "No Kernel Module Required"

Story: Sometimes you need quick I2C access without writing a kernel driver. The `i2c-dev` module exposes I2C buses as character devices (`/dev/i2c-N`). You'll write userspace programs to communicate with I2C devices directly.

📖 Background

Userspace I2C:

Your C prog or Python	<code>open("/dev/i2c-1")</code> <code>ioctl(fd, I2C_SLAVE, addr)</code> <code>read()/write()</code> or <code>i2c_smbus_*</code>
<code>i2c-dev.ko</code>	Character device layer
I2C Core	
I2C Adapter	<code>bcm2835-i2c</code> (RPi)

✅ Challenges

Challenge 27.1 — Userspace I2C in C 🍓

Write a C program to read an I2C device:

```
#include <linux/i2c-dev.h>
#include <sys/ioctl.h>
#include <fcntl.h>
#include <unistd.h>

int fd = open("/dev/i2c-1", O_RDWR);
ioctl(fd, I2C_SLAVE, 0x68);      /* Set slave address */

/* Write register address, then read */
unsigned char reg = 0x75;        /* WHO_AM_I */
write(fd, &reg, 1);
unsigned char val;
read(fd, &val, 1);
printf("WHO_AM_I = 0x%02x\n", val);
```

1. Read the WHO_AM_I register of an I2C device
2. Read multiple registers (accelerometer X, Y, Z)
3. Write a configuration register
4. Add error handling for all operations

Challenge 27.2 — I2C Scan Utility

Write a program that scans all addresses 0x03–0x77 on a bus:

1. For each address, try `ioctl(fd, I2C_SLAVE, addr)` then `read(fd, &dummy, 1)`
2. Print a grid like `i2cdetect` output
3. Mark detected vs. empty addresses

Challenge 27.3 — SMBus vs. Raw I2C

```
#include <linux/i2c-dev.h>
#include <i2c/smbus.h>

/* SMBus read (simpler, handles protocol details) */
int val = i2c_smbus_read_byte_data(fd, 0x75);

/* Raw I2C (for devices that don't follow SMBus protocol) */
struct i2c_msg msgs[2];
struct i2c_rdwr_ioctl_data data;
msgs[0].addr = 0x68; msgs[0].flags = 0; msgs[0].len = 1; msgs[0].buf = &reg;
msgs[1].addr = 0x68; msgs[1].flags = I2C_M_RD; msgs[1].len = 1; msgs[1].buf = &val;
data.msgs = msgs; data.nmsgs = 2;
ioctl(fd, I2C_RDWR, &data);
```

1. Compare SMBus read vs. raw I2C read for the same register
2. Test with an EEPROM using page-write (raw I2C needed for multi-byte write)

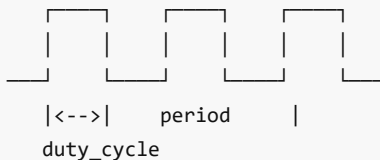
LAB 28: PWM Driver 🍓

🎯 Mission: "Pulse Width Modulation"

Story: Your project needs to control LED brightness and servo motor position. PWM (Pulse Width Modulation) varies the duty cycle of a square wave to control power delivered to a device. You must use the kernel's PWM framework from both userspace and a kernel driver.

📖 Background

PWM Signal:



Duty cycle 50% = half power
Duty cycle 100% = full power
Duty cycle 0% = off

✅ Challenges

Challenge 28.1 — PWM from Sysfs 🍓

```
# Enable hardware PWM on RPi (add to /boot/config.txt):
# dtoverlay=pwm,pin=18,func=2

# After reboot:
ls /sys/class/pwm/
echo 0 > /sys/class/pwm/pwmchip0/export
echo 1000000 > /sys/class/pwm/pwmchip0/pwm0/period      # 1ms = 1kHz
echo 500000 > /sys/class/pwm/pwmchip0/pwm0/duty_cycle    # 50%
echo 1 > /sys/class/pwm/pwmchip0/pwm0/enable
```

1. Export a PWM channel and configure period + duty cycle
2. Connect an LED to the PWM pin — observe brightness changes
3. Sweep duty cycle from 0% to 100% with a bash loop
4. For servo motors: period=20ms, duty 1-2ms

Challenge 28.2 — PWM Kernel Consumer Driver 🍓

Write a driver that requests and controls a PWM:

```
#include <linux/pwm.h>

struct pwm_device *pwm;

/* In probe: */
pwm = devm_pwm_get(&pdev->dev, NULL);
pwm_config(pwm, 500000, 1000000); /* 50% duty, 1ms period */
pwm_enable(pwm);
```

DT node:

```
my_pwm_device {
    compatible = "lab,pwm-demo";
    pwms = <&pwm 0 1000000>; /* channel 0, 1ms period */
    status = "okay";
};
```

1. Write a platform driver that controls PWM
2. Add a sysfs attribute to set duty cycle from userspace
3. Create a "breathing LED" effect using a kernel timer + PWM

LAB 29: UART Driver 🍓

🎯 Mission: "Serial Communication"

Story: A GPS module sends NMEA sentences over UART at 9600 baud. You need to receive and process this data in the kernel. The serial/tty subsystem is complex — you'll learn the layers from hardware to userspace.

📖 Background

UART/Serial Stack:

Userspace minicom, cat	/dev/ttyS0, /dev/ttyAMA0, /dev/ttyUSB0 termios settings (baud, parity, etc.)
TTY Layer	Line discipline (N_TTY default) Handles echo, canonical mode, signals
Serial Core (serial_core)	uart_port, uart_driver, uart_ops Hardware abstraction
UART Driver (your code)	Platform-specific (8250, amba-pl011, bcm2835) Handles actual TX/RX with hardware registers

✓ Challenges

Challenge 29.1 — UART Loopback Test 🍓

```
# On RPi, free up UART from console:
# In /boot/config.txt: enable_uart=1
# In /boot/cmdline.txt: remove "console=serial0,115200"
# Reboot

# Loopback: connect TX (GPIO14) to RX (GPIO15) on RPi
stty -F /dev/ttyAMA0 9600 raw -echo
echo "Hello UART" > /dev/ttyAMA0 &
cat /dev/ttyAMA0      # Should receive "Hello UART"
```

1. Configure UART speed and mode
2. Wire TX to RX for loopback
3. Send and receive data
4. Use `minicom -D /dev/ttyAMA0 -b 9600` for interactive testing

Challenge 29.2 — Serial Line Discipline Module

Write a custom line discipline that processes incoming serial data:

```
static struct tty_ldisc_ops my_ldisc_ops = {
    .owner  = THIS_MODULE,
    .num    = N_DEVELOPMENT, /* Use a free ldisc number */
    .name    = "my_ldisc",
    .open    = my_ldisc_open,
    .close   = my_ldisc_close,
    .receive_buf = my_ldisc_receive,
};
tty_register_ldisc(&my_ldisc_ops);
```

1. Register a line discipline
2. In `receive_buf()`, parse incoming data (e.g., look for NMEA sentences starting with `$GP`)
3. Log parsed data to `dmesg`
4. Attach to a serial port: `sudo ldattach N_DEV /dev/ttyAMA0`

Challenge 29.3 — serdev (Serial Device Bus) 🍓

The modern kernel approach for serial-attached devices:

```
static const struct of_device_id my_serdev_match[] = {
    { .compatible = "lab,my-serial-device" },
    { }
};

static struct serdev_device_driver my_serdev_driver = {
    .driver = {
        .name = "my_serdev",
        .of_match_table = my_serdev_match,
    },
    .probe = my_serdev_probe,
```

```
.remove = my_serdev_remove,
};
module_serdev_device_driver(my_serdev_driver);
```

1. Write a serdev driver with receive callback
2. Send data: `serdev_device_write_buf(serdev, "AT\r\n", 4, timeout)`
3. Set baud rate: `serdev_device_set_baudrate(serdev, 9600)`

LAB 30: RTC (Real-Time Clock) Driver 🍓

🎯 Mission: "Keeping Time"

Story: Your embedded system needs to maintain time across reboots. An RTC chip (often on I2C) keeps time even when the main CPU is powered off. You must write an RTC driver that integrates with the kernel's RTC framework.

📖 Background

RTC Framework:

Userspace	hwclock, timedatectl, /dev/rtc0
RTC Core (rtc-core)	struct rtc_device, rtc_class_ops Alarms, sysfs interface, chardev
RTC Driver (your code)	Your driver: read/set time from hardware Talks over I2C/SPI to RTC chip
RTC Hardware	DS3231, PCF8563, DS1307, etc.

✅ Challenges

Challenge 30.1 — Use the System RTC 🍓

```
# Read hardware clock
sudo hwclock --show
# Read via sysfs
cat /sys/class/rtc/rtc0/time
cat /sys/class/rtc/rtc0/date
# Read via chardev
sudo cat /dev/rtc0 | hexdump -C | head -3

# Set RTC from system time
sudo hwclock --systohc
# Set system time from RTC
sudo hwclock --hctosys
```

1. Read the current RTC time
2. Compare with system time (`date`)
3. Set the RTC to a specific time and verify

Challenge 30.2 — Virtual RTC Driver

Write a software-only RTC driver (no hardware needed):


```
static int my_rtc_read_time(struct device *dev, struct rtc_time *tm)
{
    /* Use system time as backing store (for demo) */
    struct timespec64 ts;
    ktime_get_real_ts64(&ts);
    rtc_time64_to_tm(ts.tv_sec + offset, tm);
    return 0;
}

static int my_rtc_set_time(struct device *dev, struct rtc_time *tm)
{
    struct timespec64 ts;
    ktime_get_real_ts64(&ts);
    offset = rtc_tm_to_time64(tm) - ts.tv_sec;
    return 0;
}

static const struct rtc_class_ops my_rtc_ops = {
    .read_time = my_rtc_read_time,
    .set_time = my_rtc_set_time,
};
```

1. Register with `devm_rtc_device_register()`
2. Verify `/dev/rtc1` appears (rtc0 is the system RTC)
3. Test with `hwclock --rtc /dev/rtc1 --show`
4. Implement alarm: `set_alarm` / `read_alarm` ops

Challenge 30.3 — I2C RTC Driver 🍓

Write a driver for DS3231 or PCF8563 RTC (connected via I2C):

1. In probe, read the seconds register to verify communication
2. Implement `read_time` by reading BCD-encoded registers 0x00-0x06
3. Convert BCD to binary with `bcd2bin()`
4. Implement `set_time` — convert binary to BCD with `bin2bcd()`
5. Register with the RTC framework

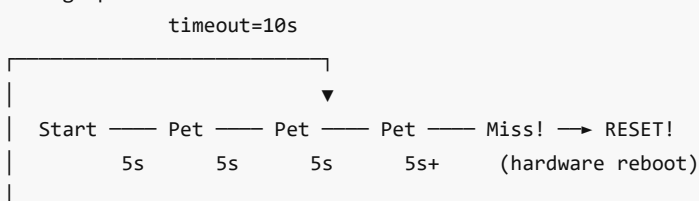
LAB 31: Watchdog Driver 🍓

🎯 Mission: "The Guardian Timer"

Story: Your embedded system runs in a remote location. If software hangs, nobody can reboot it. A **watchdog timer** automatically resets the system if the software doesn't "pet" (reset) the timer within a timeout period. You must implement a watchdog driver.

📖 Background

Watchdog Operation:



- | Normal: application pets regularly
- | Failure: application hangs → timer expires → system resets

✓ Challenges

Challenge 31.1 — Use the System Watchdog 🍓

```
# Check if watchdog is available
ls /dev/watchdog*
cat /sys/class/watchdog/watchdog0/identity
cat /sys/class/watchdog/watchdog0/timeout

# Pet the watchdog (keeps system alive)
# WARNING: opening /dev/watchdog starts the timer!
# If you don't pet it, system WILL reboot!
sudo sh -c 'echo -n "V" > /dev/watchdog' # Magic close (disarms)
```

1. Check watchdog identity and timeout
2. Understand the "magic close" feature (`V` character disables on close)
3. Read `nowayout` status — if set, watchdog cannot be stopped!

Challenge 31.2 — Software Watchdog Driver

Write a virtual watchdog driver:

```
static int my_wdt_start(struct watchdog_device *wdd)
{
    mod_timer(&wdt_timer, jiffies + wdd->timeout * HZ);
    pr_info("[wdt] Started, timeout=%ds\n", wdd->timeout);
    return 0;
}

static int my_wdt_ping(struct watchdog_device *wdd)
{
    mod_timer(&wdt_timer, jiffies + wdd->timeout * HZ);
    return 0;
}

static const struct watchdog_ops my_wdt_ops = {
    .owner = THIS_MODULE,
    .start = my_wdt_start,
    .stop = my_wdt_stop,
    .ping = my_wdt_ping,
};

static struct watchdog_device my_wdt = {
    .info = &my_wdt_info,
    .ops = &my_wdt_ops,
    .timeout = 10,
    .min_timeout = 1,
    .max_timeout = 60,
};
```

1. Register with `watchdog_register_device()`
2. When timer expires, log a warning (don't actually reboot for safety!)
3. Test: `echo 1 > /dev/watchdog1` to start, then stop petting

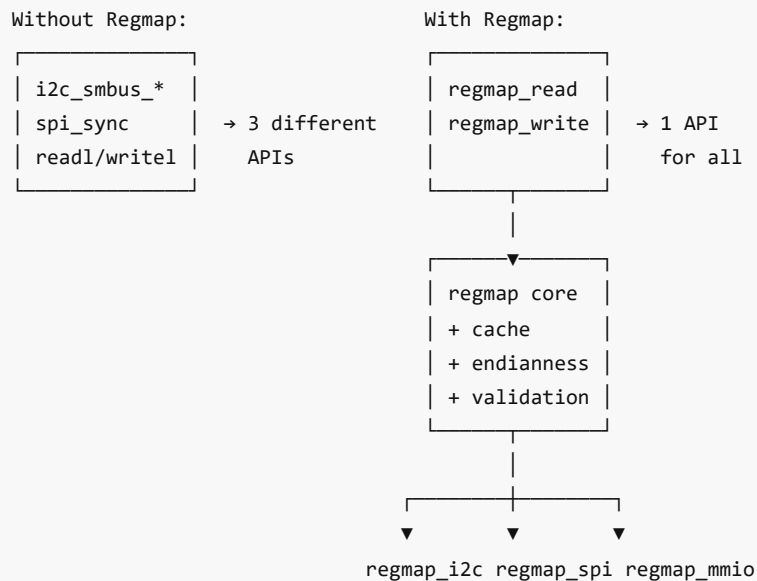
4. Implement `set_timeout` to allow userspace to change timeout

LAB 32: Regmap API 🍓

🎯 Mission: "One API to Rule Them All"

Story: You've written I2C reads, SPI reads, and MMIO reads — all slightly different APIs. The **regmap** framework provides a single, unified API for register access regardless of the bus. It also adds caching, endianness handling, and range checking for free.

📖 Background



✅ Challenges

Challenge 32.1 — Regmap with I2C

Convert an I2C driver to use regmap:

```
static const struct regmap_config my_regmap_config = {
    .reg_bits = 8,
    .val_bits = 8,
    .max_register = 0xFF,
    .cache_type = REGCACHE_RBTREE,
};

/* In probe: */
struct regmap *regmap = devm_regmap_init_i2c(client, &my_regmap_config);

/* Read/Write: */
unsigned int val;
regmap_read(regmap, 0x75, &val);      /* Read WHO_AM_I */
regmap_write(regmap, 0x6B, 0x00);     /* Wake up device */
regmap_update_bits(regmap, 0x1C, 0x18, 0x10); /* Modify specific bits */
```

1. Initialize regmap for your I2C device
2. Read device ID register
3. Use `regmap_update_bits()` to modify a field without read-modify-write

4. Enable caching and observe that repeated reads hit cache

Challenge 32.2 — Regmap with SPI

```
struct regmap *regmap = devm_regmap_init_spi(spi, &my_regmap_config);
```

1. Switch the same logical driver from I2C to SPI regmap — only init changes
2. Add readable/writable register tables:

```
static bool my_readable(struct device *dev, unsigned int reg) {  
    return reg <= 0x75;  
}  
.  
.  
.  
.readable_reg = my_readable,
```

Challenge 32.3 — Regmap Bulk and Field Operations

1. Bulk read: `regmap_bulk_read(regmap, start_reg, buf, count)`
2. Register fields:

```
REG_FIELD(0x1C, 3, 4); /* register 0x1C, bits 3:4 */  
struct regmap_field *field = devm_regmap_field_alloc(dev, regmap, field_def);  
regmap_field_read(field, &val);
```

3. Use `regcache_sync()` after power resume

LAB 33: IIO (Industrial I/O) Driver 🍓

🎯 Mission: "Sensors Done Right"

Story: Your accelerometer/temperature/ADC sensor needs a proper kernel interface. The **IIO (Industrial I/O)** subsystem is the standard framework for sensors, ADCs, DACs, and other measurement devices. It provides buffered/triggered data capture, sysfs attributes, and a chardev for high-speed streaming.

📖 Background

IIO Framework:

Userspace
cat /sys/bus/iio/devices/iio:device0/ in_accel_x_raw libiio, iio_info, iio_readdev
IIO Core
iio_device, iio_chan_spec Triggered buffers, events
IIO Driver (your code)
.read_raw(), .write_raw() Channel definitions
Bus Layer (I2C/SPI/platform)

✓ Challenges

Challenge 33.1 — Explore IIO Devices

```
# Install IIO tools
sudo apt install -y libiio-utils

# List IIO devices
iio_info
ls /sys/bus/iio/devices/

# Read a channel (if an IIO device exists)
cat /sys/bus/iio/devices/iio:device0/in_temp_input
cat /sys/bus/iio/devices/iio:device0/name
```

1. List existing IIO devices on your system
2. Read available channels and their raw/scale values
3. Understand the naming: `in_<type>_<index>_raw`, `_scale`, `_offset`

Challenge 33.2 — Simple IIO Driver (Virtual Sensor) 🍓

Write an IIO driver that produces fake sensor data:

```
static const struct iio_chan_spec my_channels[] = {
    {
        .type = IIO_TEMP,
        .info_mask_separate = BIT(IIO_CHAN_INFO_RAW) | BIT(IIO_CHAN_INFO_SCALE),
    },
    {
        .type = IIO_ACCEL,
        .modified = 1,
        .channel2 = IIO_MOD_X,
        .info_mask_separate = BIT(IIO_CHAN_INFO_RAW),
        .info_mask_shared_by_type = BIT(IIO_CHAN_INFO_SCALE),
    },
    /* Y, Z channels similar */
};

static int my_read_raw(struct iio_dev *indio_dev,
                      struct iio_chan_spec const *chan,
                      int *val, int *val2, long mask)
{
    switch (mask) {
    case IIO_CHAN_INFO_RAW:
        if (chan->type == IIO_TEMP)
            *val = 2500; /* 25.00°C */
        else
            *val = get_random_u32() % 2000 - 1000;
        return IIO_VAL_INT;
    case IIO_CHAN_INFO_SCALE:
        *val = 0;
        *val2 = 10000; /* 0.01 */
        return IIO_VAL_INT_PLUS_MICRO;
    }
    return -EINVAL;
}
```

1. Register an IIO device with temperature and accelerometer channels
2. Read via sysfs: `cat /sys/bus/iio/devices/iio:deviceN/in_temp_raw`
3. Calculate real value: `raw * scale`
4. Add writable calibration offset

Challenge 33.3 — IIO Triggered Buffer 🍓

Add buffered/streaming mode:

1. Set up an IIO trigger: `devm_iio_trigger_alloc()`
2. Configure triggered buffer: `devm_iio_triggered_buffer_setup()`
3. In trigger handler, push data: `iio_push_to_buffers_with_timestamp()`
4. Read from userspace: `cat /dev/iio:device0` or `iio_readdev -t -b 256`

LAB 34: USB Driver

🔧 Mission: "Universal Serial Bus"

Story: You've plugged in a custom USB device. The kernel needs a driver to communicate with it. USB is the most common bus in computing — understanding the USB driver model is essential for any Linux developer.

📖 Background

USB Stack:

Userspace	libusb, /dev/bus/usb/
USB Device Driver (your code)	usb_driver, probe/disconnect Endpoints, URBs, pipes
USB Core	usb_register(), usb_submit_urb()
Host Ctrl. Driver (HCD)	EHCI, xHCI, OHCI, dwc_otg (RPI)
Hardware	USB port, cable, device

USB Transfer Types:

- Control — device setup, configuration
- Bulk — large data (storage, printers)
- Interrupt — small, periodic (HID, keyboard)
- Isochronous — streaming (audio, video)

✅ Challenges

Challenge 34.1 — Explore USB Devices

```
# List USB devices
lsusb
lsusb -v -d VEND:PROD # Detailed view of specific device
lsusb -t               # Tree view

# USB filesystem
```

```
ls /sys/bus/usb/devices/
cat /sys/bus/usb/devices/1-1/idVendor
cat /sys/bus/usb/devices/1-1/idProduct
cat /sys/bus/usb/devices/1-1/manufacturer
```

1. List all USB devices and identify vendor/product IDs
2. Find the USB tree topology
3. Read device descriptors from sysfs
4. Use `usbmon` to sniff USB traffic: `sudo modprobe usbmon; sudo cat /sys/kernel/debug/usb/usbmon/0u`

Challenge 34.2 — Simple USB Driver

Write a USB driver that binds to a specific device:

```
static const struct usb_device_id my_usb_id[] = {
    { USB_DEVICE(0x1234, 0x5678) }, /* Your device VID:PID */
    { }
};
MODULE_DEVICE_TABLE(usb, my_usb_id);

static int my_usb_probe(struct usb_interface *intf,
                       const struct usb_device_id *id)
{
    struct usb_device *udev = interface_to_usbdev(intf);
    dev_info(&intf->dev, "USB device connected: %s %s\n",
            udev->manufacturer, udev->product);
    /* Find endpoints */
    struct usb_endpoint_descriptor *ep;
    ep = &intf->cur_altsetting->endpoint[0].desc;
    dev_info(&intf->dev, "EP addr=0x%02x, type=%d\n",
            ep->bEndpointAddress, usb_endpoint_type(ep));
    return 0;
}

static struct usb_driver my_usb_driver = {
    .name      = "my_usb_driver",
    .probe     = my_usb_probe,
    .disconnect = my_usb_disconnect,
    .id_table  = my_usb_id,
};
module_usb_driver(my_usb_driver);
```

1. Bind to a specific USB device (use a USB flash drive for testing)
2. Enumerate interfaces and endpoints
3. Log device speed (`udev->speed`)

Challenge 34.3 — USB Bulk Transfer

1. Allocate a URB: `usb_alloc_urb(0, GFP_KERNEL)`
2. Fill bulk URB: `usb_fill_bulk_urb()`
3. Submit: `usb_submit_urb(urb, GFP_KERNEL)`
4. Handle completion callback
5. Alternative: use `usb_bulk_msg()` for synchronous transfer

LAB 35: PCI Driver

Mission: "The Desktop Bus"

Story: PCI/PCIe is the high-speed bus for GPUs, network cards, NVMe drives, and more. Unlike platform devices, PCI devices are auto-discovered. You must write a PCI driver that probes when a matching device is found.

Background

PCI Stack:

PCI Driver (your code)	pci_driver, probe/remove DMA, MMIO, MSI interrupts
PCI Core	pci_register_driver() Resource management, config space
PCI Bus Driver	Enumeration, BAR allocation
Hardware	PCIe slot, config space, BARs

Challenges

Challenge 35.1 — Explore PCI Devices

```
# List PCI devices
lspci
lspci -v          # Verbose
lspci -nn         # Show vendor:device codes
lspci -s 00:02.0 -v # Specific device

# Sysfs
ls /sys/bus/pci/devices/
cat /sys/bus/pci/devices/0000:00:02.0/vendor
cat /sys/bus/pci/devices/0000:00:02.0/device
# Config space
hexdump -C /sys/bus/pci/devices/0000:00:02.0/config | head
```

1. List all PCI devices with vendor/device IDs
2. Find BARs (Base Address Registers) for a device
3. Read the config space header (64 bytes)
4. Identify which driver is bound to each device

Challenge 35.2 — PCI Skeleton Driver

```
static const struct pci_device_id my_pci_ids[] = {
    { PCI_DEVICE(0x8086, 0x100e) }, /* Intel e1000 (QEMU default NIC) */
    { }
};
MODULE_DEVICE_TABLE(pci, my_pci_ids);

static int my_pci_probe(struct pci_dev *pdev, const struct pci_device_id *id)
{
    int ret;
    void __iomem *bar0;
```



```

ret = pcim_enable_device(pdev);
if (ret) return ret;

ret = pcim_iomap_regions(pdev, BIT(0), "my_pci");
if (ret) return ret;

bar0 = pcim_iomap_table(pdev)[0];
dev_info(&pdev->dev, "BAR0 mapped at %p, size=%llu\n",
        bar0, pci_resource_len(pdev, 0));

/* Read a register */
u32 val = ioread32(bar0 + 0x00); /* Device status */
dev_info(&pdev->dev, "Status register: 0x%08x\n", val);

pci_set_master(pdev); /* Enable bus mastering for DMA */
return 0;
}

static struct pci_driver my_pci_driver = {
    .name      = "my_pci_driver",
    .id_table  = my_pci_ids,
    .probe     = my_pci_probe,
    .remove    = my_pci_remove,
};
module_pci_driver(my_pci_driver);

```

1. Test in QEMU with the default e1000 NIC (unbind from real driver first)
2. Map BAR0 and read status registers
3. Enable bus mastering for DMA capability

Challenge 35.3 — PCI DMA and MSI Interrupts

1. Allocate DMA-coherent buffer: `dma_alloc_coherent()`
2. Set up MSI/MSI-X interrupts: `pci_alloc_irq_vectors(pdev, 1, 1, PCI_IRQ_MSI)`
3. Request IRQ: `pci_irq_vector(pdev, 0) → request_irq()`
4. Understand `pci_set_dma_mask()` for 32/64-bit DMA addressing

LAB 36: Block Driver

Mission: "Creating a Virtual Disk"

Story: Block drivers handle storage devices — hard drives, SSDs, RAM disks. Unlike character devices (byte stream), block devices handle data in fixed-size blocks. You must create a RAM-backed block device that appears as `/dev/myblk`.

Background

Block Layer:

Filesystem	ext4, xfs, btrfs
VFS	open, read, write, sync
Block Layer	I/O scheduler, bio, request queues

Block Driver (your code)	Your code: process requests, access storage
Storage	RAM (ramdisk), actual disk, network

✓ Challenges

Challenge 36.1 — RAM Block Device (blk-mq)

Write a block driver using the modern multi-queue (blk-mq) API:

```
static struct blk_mq_ops my_blk_ops = {
    .queue_rq = my_queue_rq,
};

static blk_status_t my_queue_rq(struct blk_mq_hw_ctx *hctx,
                                const struct blk_mq_queue_data *bd)
{
    struct request *rq = bd->rq;
    struct bio_vec bvec;
    struct req_iterator iter;

    blk_mq_start_request(rq);

    rq_for_each_segment(bvec, rq, iter) {
        void *buf = page_address(bvec.bv_page) + bvec.bv_offset;
        sector_t sector = iter.iter.bi_sector;
        unsigned long offset = sector << 9; /* sector * 512 */

        if (rq_data_dir(rq) == READ)
            memcpy(buf, ramdisk + offset, bvec.bv_len);
        else
            memcpy(ramdisk + offset, buf, bvec.bv_len);
    }

    blk_mq_end_request(rq, BLK_STS_OK);
    return BLK_STS_OK;
}
```

1. Allocate a RAM buffer (e.g., 16 MB using `vmalloc`)
2. Create a gendisk with `blk_alloc_disk()` or `blk_mq_alloc_disk()`
3. Set capacity: `set_capacity(disk, nsectors)`
4. Register: `add_disk(disk)`

Challenge 36.2 — Format and Mount

```
# After loading module:
sudo mkfs.ext4 /dev/myblk
sudo mkdir -p /mnt/myblk
sudo mount /dev/myblk /mnt/myblk
echo "Hello Block Device" | sudo tee /mnt/myblk/test.txt
cat /mnt/myblk/test.txt
sudo umount /mnt/myblk
```

1. Create a filesystem on your block device

2. Mount it and create files
3. Verify data persists across unmount/remount (while module stays loaded)
4. Check `lsblk` and `cat /proc/partitions`

Challenge 36.3 — Block Device with Partitions

1. Set `disk->flags |= GENHD_FL_NO_PART` to disable, or remove it to enable partitions
2. Use `fdisk /dev/myblk` to create partitions
3. Observe `/dev/myblk1`, `/dev/myblk2` appear

LAB 37: Process Management

Mission: "Inside the Scheduler"

Story: The kernel's process scheduler decides which of thousands of processes gets CPU time. You must understand the `task_struct`, scheduling classes, and how to observe and influence process scheduling from kernel space.

Background

Scheduling Classes (priority order):

Stop class (highest)	Migration threads
Deadline (SCHED_DEADLINE)	EDF scheduler
RT (SCHED_FIFO/SCHED_RR)	Real-time, fixed priority 1-99
CFS (SCHED_NORMAL)	Fair scheduling, nice -20 to +19
Idle (SCHED_IDLE)	Only when nothing else to run

Challenges

Challenge 37.1 — Explore `task_struct`

Write a module that walks the process list:

```
#include <linux/sched/signal.h>

struct task_struct *task;

for_each_process(task) {
    pr_info("[proc] PID=%d TGID=%d comm=%s state=%d prio=%d "
           "policy=%d parent=%s\n",
           task->pid, task->tgid, task->comm,
           task_state_index(task), task->prio,
           task->policy, task->parent->comm);
}
```

1. List all processes with PID, name, state, priority
2. Find your own process (`current`) and print its details
3. Walk the parent chain from current to init (PID 1)
4. Count processes per state (running, sleeping, stopped, zombie)

Challenge 37.2 — Scheduling Experiments

```
# Change scheduler policy
sudo chrt -f 50 ./my_program      # SCHED_FIFO, priority 50
sudo chrt -r 50 ./my_program      # SCHED_RR, priority 50
sudo chrt -d --sched-period 1000000 --sched-runtime 5000000 ./my_program # SCHED_DEADLINE

# Change nice value
nice -n -10 ./my_program
renice -n 5 -p <PID>

# View scheduling info
chrt -p <PID>
cat /proc/<PID>/sched
cat /proc/<PID>/status | grep -i policy
```

1. Run CPU-intensive tasks with different policies and compare throughput
2. Observe CFS vruntime: `cat /proc/<PID>/sched | grep vruntime`
3. Test RT priority: does SCHED_FIFO starve normal tasks?
4. Explore `/proc/sys/kernel/sched_*` tunables

Challenge 37.3 — Module to Set Process Priority

Write a module with a `/proc` interface that lets you change a process's scheduler:

1. Accept input: `echo "PID POLICY PRIORITY" > /proc/sched_control`
2. Use `sched_setscheduler()` to change the policy
3. Read `/proc/sched_control` to show current process info
4. Handle errors (invalid PID, insufficient permissions)

LAB 38: RT Linux (PREEMPT_RT)

Mission: "Real-Time Determinism"

Story: Your control system needs guaranteed response times of under 100 microseconds. Standard Linux can't promise this — the kernel can delay interrupts and disable preemption. **PREEMPT_RT** patches make Linux a hard real-time OS by making almost everything preemptible.

Background

Preemption Models:

PREEMPT_NONE	– Server workloads
Minimal preemption, best throughput	
PREEMPT_VOLUNTARY	– Desktop default
Explicit preemption points added	
PREEMPT	– Low-latency
Preemptible kernel (except spinlocks)	
PREEMPT_RT (full)	– Hard real-time
Nearly everything preemptible	
Spinlocks become sleeping locks	
Threaded IRQs mandatory	

✓ Challenges

Challenge 38.1 — Check RT Status

```
# Is your kernel RT?
uname -a | grep -i rt
cat /sys/kernel/realtime 2>/dev/null && echo "RT kernel!" || echo "Not RT"
zcat /proc/config.gz | grep PREEMPT

# Check preemption model
cat /proc/sys/kernel/sched_rt_runtime_us
cat /proc/sys/kernel/sched_rt_period_us
```

1. Determine your kernel's preemption model
2. Check RT throttling settings (default: RT tasks get 950ms per 1000ms)
3. View threaded IRQs: `ps -eo pid,cls,rtprio,ni,comm | grep irq`

Challenge 38.2 — Build an RT Kernel 🍓

```
# Get kernel source
wget https://cdn.kernel.org/pub/linux/kernel/v6.x/linux-6.6.tar.xz
wget https://cdn.kernel.org/pub/linux/kernel/projects/rt/6.6/patch-6.6-rt15.patch.xz

# Apply RT patch
xzcat patch-6.6-rt15.patch.xz | patch -p1

# Configure
make menuconfig
# → General setup → Preemption Model → Fully Preemptible (RT)
# → Kernel hacking → Tracers → enable ftrace, irqsoff tracer
```

1. Download kernel source + RT patch
2. Apply the patch and configure for full RT
3. Build and boot the RT kernel
4. Verify with `uname -a` and `/sys/kernel/realtime`

Challenge 38.3 — Latency Testing

```
# Install cyclicttest
sudo apt install -y rt-tests

# Run latency test (needs RT kernel for best results)
sudo cyclicttest -m -Sp90 -i200 -h400 -q -D 60
# -m: lock memory, -S: SMP, -p90: RT priority 90
# -i200: 200µs interval, -D 60: run 60 seconds

# View histogram
sudo cyclicttest -m -Sp90 -i200 -h400 -q -D 60 > output.txt
grep "Max Latencies" output.txt
```

1. Run `cyclicttest` on non-RT and RT kernels, compare max latencies
2. Introduce load: `stress -c $(nproc) -i 4 -m 2 --timeout 60`

3. Compare latencies under load
4. Target: max latency < 100µs on RT kernel

Challenge 38.4 — RT-Safe Kernel Programming

Write a module that demonstrates RT-safe practices:

1. Use `raw_spinlock_t` instead of `spinlock_t` (not converted to mutex on RT)
2. Measure interrupt latency using `ktime_get()` in an IRQ handler
3. Show that `local_irq_disable()` duration is minimized on RT
4. Demonstrate priority inheritance with `rt_mutex`

LAB 39: Audio Subsystem (ALSA/ASoC)

Mission: "Making Sound"

Story: Your board needs audio output. The kernel's audio stack (ALSA + ASoC) handles everything from USB headsets to SoC audio codecs. You must understand the layers and write a simple ASoC driver.

Background

Audio Stack:

PulseAudio	or PipeWire (user-space mixer/routing)
ALSA lib	libasound2 (user-space library)
ALSA Core	snd_card, snd_pcm, snd_control
ASoC (for embedded)	Machine + Codec + Platform drivers DAI links, DAPM widgets
Hardware	I2S, codec chip, DAC/ADC

Challenges

Challenge 39.1 — Explore ALSA 🍓

```
# List sound cards
cat /proc/asound/cards
aplay -l
arecord -l

# Play a test tone
speaker-test -t sine -f 440 -D default

# Record and play back
arecord -d 5 -f cd test.wav
aplay test.wav

# ALSA mixer
alsamixer
```

```
amixer scontrols
amixer sset 'Master' 80%
```

1. List audio devices and their capabilities
2. Play a sine wave tone
3. Record and play back audio
4. Examine controls with `amixer`

Challenge 39.2 — ASoC Machine Driver Overview

Study the ASoC architecture:

1. Find the machine driver for your board:

```
ls /sys/bus/platform/drivers/ | grep -i sound
modinfo snd_bcm2835    # RPi audio driver
```

2. Read the source for a simple machine driver in `sound/soc/bcm/`
3. Understand DAI (Digital Audio Interface) links
4. Trace the path: application → ALSA → ASoC → I2S → codec → speaker

Challenge 39.3 — Virtual Sound Card (Dummy)

```
# Load the dummy sound driver
sudo modprobe snd-dummy
cat /proc/asound/cards
# Use the dummy card for testing
aplay -D hw:Dummy,0 test.wav
```

1. Load the dummy sound driver and verify it appears
2. Study `sound/drivers/dummy.c` in kernel source
3. Understand how `snd_card_new()`, `snd_pcm_new()`, and PCM ops work
4. Write a minimal skeleton that registers a sound card (even without real PCM)

LAB 40: Display Subsystem (DRM/KMS) 🍓

🎯 Mission: "Pixels on Screen"

Story: You need to get graphics output from your board. The **DRM (Direct Rendering Manager) / KMS (Kernel Mode Setting)** subsystem handles display output, mode setting, framebuffers, and GPU access. This is the kernel foundation under X11 and Wayland.

📖 Background

Display Stack:

Wayland/X11	Compositor / Display server
Mesa / GL	3D rendering, shader compilation
libdrm	Userspace DRM library
DRM/KMS Core	CRTC, encoder, connector, plane GEM (buffer management)

DRM Driver (your code)	i915 (Intel), amdgpu, vc4 (RPi)
Hardware	GPU, display controller, HDMI/DSI/LVDS

KMS Objects:

Framebuffer → Plane → CRTC → Encoder → Connector → Monitor
 (pixels) (layer) (scanout) (signal) (port) (display)

✓ Challenges

Challenge 40.1 — Explore DRM/KMS 🍓

```
# List DRM devices
ls /dev/dri/
cat /sys/class/drm/card0/device/driver/module/name

# DRM info tool
sudo apt install -y libdrm-tests drm-info
drm_info

# List connectors, encoders, CRTCs
sudo apt install -y modetest # from libdrm-tests
modetest -M vc4 # RPi
# or
modetest -M i915 # Intel

# Read current mode
cat /sys/class/drm/card0-HDMI-A-1/modes
cat /sys/class/drm/card0-HDMI-A-1/status
```

1. List DRM devices and their connectors
2. Find current display resolution and refresh rate
3. List supported display modes
4. Identify the DRM driver in use

Challenge 40.2 — Direct Framebuffer Access

Write a userspace program that draws directly to a DRM framebuffer:

1. Open `/dev/dri/card0`
2. Use `drmModeGetResources()` to find connectors/CRTCs
3. Create a dumb buffer: `drmIoctl(fd, DRM_IOCTL_MODE_CREATE_DUMB, ...)`
4. Map it: `mmap()` the buffer
5. Draw pixels (fill with color, draw shapes)
6. Set mode: `drmModeSetCrtc()`
7. This is the lowest-level way to put pixels on screen

Challenge 40.3 — Study DRM Driver Structure

1. Read `drivers/gpu/drm/tiny/simpliedrm.c` — a minimal DRM driver
2. Understand: `drm_driver`, `drm_simple_display_pipe_funcs`
3. Study the RPi driver: `drivers/gpu/drm/vc4/`
4. Understand GEM (Graphics Execution Manager) for buffer management
5. Write notes on how a DRM driver registers: `drm_dev_alloc()`, `drm_dev_register()`

LAB 41: Camera Subsystem (V4L2) 🍓

🎯 Mission: "Capturing Images"

Story: A camera module is connected to your Raspberry Pi. The **V4L2 (Video4Linux2)** framework handles camera capture, video encoding, and image processing in the kernel. You must understand V4L2, capture frames, and explore driver structure.

📖 Background

V4L2 Stack:

Userspace	v4l2-ctl, ffmpeg, gstreamer /dev/video0
V4L2 Core	videobuf2, v4l2_device, v4l2_subdev Controls, formats, streaming
V4L2 Driver (your code)	Your driver: register video_device Implement vidioc_* ioctl handlers
Media Ctrl Framework	media_device, entities, links Pipeline management
Camera/ISP Hardware	CSI-2 receiver, ISP, sensor

✅ Challenges

Challenge 41.1 — Capture Images 🍓

```
# On RPi, enable camera:
sudo raspi-config # Interface Options → Camera → Enable

# Install V4L2 tools
sudo apt install -y v4l-utils ffmpeg

# List video devices
v4l2-ctl --list-devices

# Query capabilities
v4l2-ctl -d /dev/video0 --all

# List supported formats
v4l2-ctl -d /dev/video0 --list-formats-ext

# Capture a single frame
v4l2-ctl -d /dev/video0 --set-fmt-video=width=640,height=480,pixelformat=YUYV \
    --stream-mmap --stream-count=1 --stream-to=frame.raw

# Capture with ffmpeg
ffmpeg -f v4l2 -video_size 640x480 -i /dev/video0 -frames:v 1 photo.jpg
```

1. List video devices and their capabilities

2. Check supported pixel formats and resolutions
3. Capture a single frame and view it
4. Capture a 10-second video

Challenge 41.2 — V4L2 Controls

```
# List controls
v4l2-ctl -d /dev/video0 --list-ctrls

# Set brightness
v4l2-ctl -d /dev/video0 --set-ctrl=brightness=128

# Get control value
v4l2-ctl -d /dev/video0 --get-ctrl=brightness
```

1. List all available controls (brightness, contrast, exposure, etc.)
2. Adjust controls and observe the effect
3. Write a C program using `ioctl(VIDIOC_S_CTRL)` to set controls

Challenge 41.3 — V4L2 Capture in C

Write a userspace V4L2 capture program:

1. Open `/dev/video0`
2. Query capabilities: `ioctl(VIDIOC_QUERYCAP)`
3. Set format: `ioctl(VIDIOC_S_FMT)` — resolution, pixel format
4. Request buffers: `ioctl(VIDIOC_REQBUFS)` (MMAP method)
5. Queue buffers: `ioctl(VIDIOC_QBUF)`
6. Start streaming: `ioctl(VIDIOC_STREAMON)`
7. Dequeue filled buffer: `ioctl(VIDIOC_DQBUF)`
8. Save the frame data to a file
9. Stop streaming: `ioctl(VIDIOC_STREAMOFF)`

Challenge 41.4 — Media Controller

```
# Explore the media controller graph
media-ctl -d /dev/media0 --print-topology

# Set up a pipeline (RPi)
media-ctl -d /dev/media0 -l "'imx219 10-0010':0->'unicam-image':0[1]"
```

1. Install `media-ctl` and explore the device topology
2. Understand entities, pads, and links
3. Trace the path from sensor to video node



MINI-PROJECT 5 — I2C Sensor Dashboard



Objective: Build a complete sensor monitoring system using I2C.

Requirements:

1. Write an I2C kernel driver for a temperature/humidity sensor (e.g., SHT31, BME280)
2. Use regmap for register access
3. Register as an IIO device with proper channels
4. Create a debugfs directory showing raw register dumps
5. Write a userspace tool that reads IIO sysfs and displays data

6. Support configurable sample rate via sysfs

Deliverables:

- Kernel module (I2C driver + IIO registration)
 - Device tree overlay
 - Userspace monitoring tool
 - Documentation showing real sensor readings
-



MINI-PROJECT 6 — GPIO Home Automation Controller



Objective: Build a kernel-based GPIO controller with input events and PWM.

Requirements:

1. Platform driver with DT binding
2. Control 4 GPIOs (2 output LEDs, 1 PWM LED, 1 input button)
3. Button registered as input device (visible to `evtest`)
4. PWM LED with adjustable brightness via sysfs
5. Watchdog timer that toggles a "heartbeat" LED
6. `procfs` status page showing all GPIO states

Deliverables:

- Platform driver module
 - DT overlay for RPi
 - Test scripts
 - Wiring diagram (ASCII art in README)
-



MINI-PROJECT 7 — Virtual Block Device with Encryption

Objective: Create a RAM block device that optionally XOR-encrypts data.

Requirements:

1. blk-mq based block driver (16 MB RAM disk)
2. Module parameter to enable/disable encryption
3. Simple XOR cipher with configurable key (for educational purposes only)
4. sysfs attributes showing I/O statistics (reads, writes, bytes)
5. Support partitioning
6. Can be formatted with ext4 and mounted

Deliverables:

- Block driver module
 - Makefile
 - Test script: create, mount, write, read, verify
-



EXTENDED CAPSTONE — Multi-Bus Sensor Platform



Build a production-quality sensor platform driver that demonstrates mastery of Linux kernel driver development.

Overview

Create a platform driver that manages sensors on multiple buses (I2C, SPI, GPIO), exposes data through standard subsystems (IIO, input, hwmon), and includes proper device tree bindings, power management, and testing.

Phase 1 — Platform Driver Foundation

1. Write a platform driver with `of_match_table`
2. Parse DT properties: number of sensors, bus types, addresses
3. Use `devm_*` APIs exclusively for resource management
4. Implement proper error handling with goto-chain cleanup
5. Add PM ops: `suspend`, `resume`, `runtime_suspend`, `runtime_resume`

Phase 2 — I2C Sensor Integration

1. Add an I2C client sub-driver for temperature sensor
2. Use regmap for all register access with caching
3. Register as IIO device with temperature channel
4. Add triggered buffer support for continuous monitoring
5. Implement calibration via writable sysfs attributes

Phase 3 — SPI Sensor Integration

1. Add SPI client for an accelerometer
2. Register IIO channels: `in_accel_x/y/z_raw`
3. Add high-speed DMA transfers for bulk reads
4. Implement IIO trigger (data-ready interrupt from sensor)

Phase 4 — GPIO and Input Integration

1. GPIO button registered as input device
2. GPIO LED controlled via LED subsystem (`struct led_classdev`)
3. PWM buzzer for alerts (configurable frequency via sysfs)
4. IRQ-driven button with debounce

Phase 5 — System Integration

1. Watchdog: reset if sensor reads stall for 30 seconds
2. Debugfs: raw register dumps, driver statistics, error counts
3. procfs: human-readable sensor summary
4. sysfs: per-sensor attributes, calibration, sample rate
5. RTC alarm: wake the system for periodic readings (if in low-power mode)

Phase 6 — Testing and Documentation

1. Write a comprehensive test script:
 - Load/unload module in loops (no memory leaks)
 - Stress test with concurrent readers
 - Suspend/resume cycles
 - Invalid input handling
2. Test on QEMU (with `i2c-stub` and dummy SPI)
3. Test on Raspberry Pi with real sensors
4. Write device tree binding documentation (YAML schema format)
5. Generate kernel-doc from source comments

Evaluation Criteria

- All subsystem registrations work correctly (IIO, input, LED, watchdog)
- Regmap caching reduces I2C/SPI traffic

- No resource leaks on load/unload cycles
- Works on both x86_64 VM (with stubs) and Raspberry Pi (🍓)
- Driver survives `modprobe / rmmmod` stress testing
- Proper DT binding document exists
- PM ops correctly save/restore device state

End of Linux Kernel Programming